Compilers

Nathan Warner

Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

Introduction

- **Compilers**: In its most general form, a compiler is a program that accepts as input a program text in a certain language and produces as output a program text in another language, while preserving the meaning of that text

- **Translation, source language, target language, and implementation language**: This process is called translation, as it would be if the texts were in natural languages. Almost all compilers translate from one input language, the source language, to one output language, the target language, only. One normally expects the source and target language to differ greatly: the source language could be C and the target language might be machine code for the Pentium processor series. The language the compiler itself is written in is the implementation language.

  To obtain the translated program, we run a compiler, which is just another program whose input is a file with the format of a program source text and whose output is a file with the format of executable code

- **Bootstrapping**: When the source language is also the implementation language and the source text to be compiled is actually a new version of the compiler itself, the process is called bootstrapping.

- **Front and back end**: The part of a compiler that performs the analysis of the source language text is called the front-end, and the part that does the target language synthesis is the back-end

  If the compiler has a very clean design, the front-end is totally unaware of the target language and the back-end is totally unaware of the source language, the only thing they have in common is knowledge of the semantic representation

- **Parse tree / syntax tree**: The **syntax tree** of a program text is a data structure which shows precisely how the various segments of the program text are to be viewed in terms of the grammar. The syntax tree can be obtained through a process called "parsing"

  Parsing is the process of structuring a text according to a given grammar. For this reason, syntax trees are also called **parse trees**; we will use the terms interchangeably, with a slight preference for "parse tree" when the emphasis is on the actual parsing. Conversely, parsing is also called syntax analysis

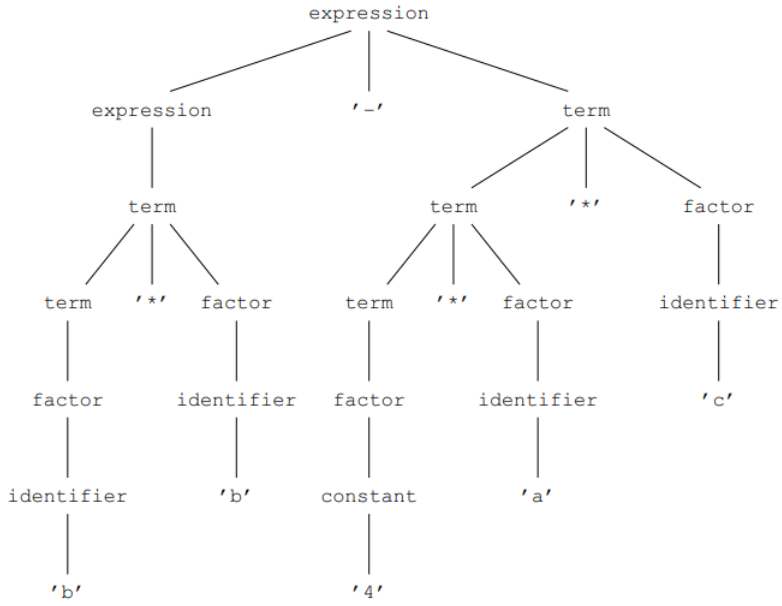
- **Abstract syntax tree (AST)**: The exact form of the parse tree as required by the grammar is often not the most convenient one for further processing, so usually a modified form of it is used, called an abstract syntax tree, or AST. Detailed information about the semantics can be attached to the nodes in this tree through annotations, which are stored in additional data fields in the nodes; hence the term annotated abstract syntax tree. Since unannotated ASTs are of limited use, ASTs are always more or less annotated in practice, and the abbreviation "AST" is used also for annotated ASTs.

- **Lexical analysis**: Usually the grammar of a programming language is not specified in terms of input characters but of input "tokens". Input tokens may be and sometimes must be separated by white space, which is otherwise ignored. So before feeding the input program text to the parser, it must be divided into tokens. Doing so is the task of the lexical analyzer; the activity itself is sometimes called "to tokenize", but the literary value of that word is doubtful.

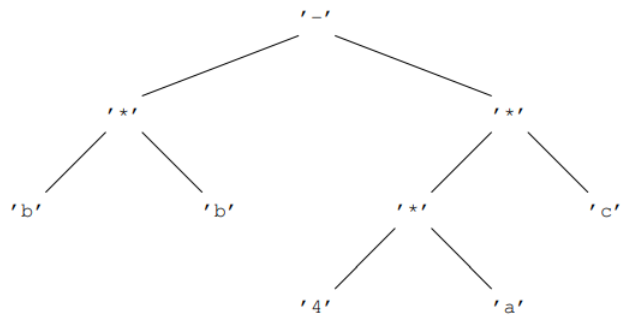
- **Example grammer, parse tree, and AST**: Consider the grammer

$$E \to E + T \mid E - T \mid T$$
$$T \to T * F \mid T/F \mid F$$
$$F \to \text{identifier} \mid \text{constant} \mid (E),$$

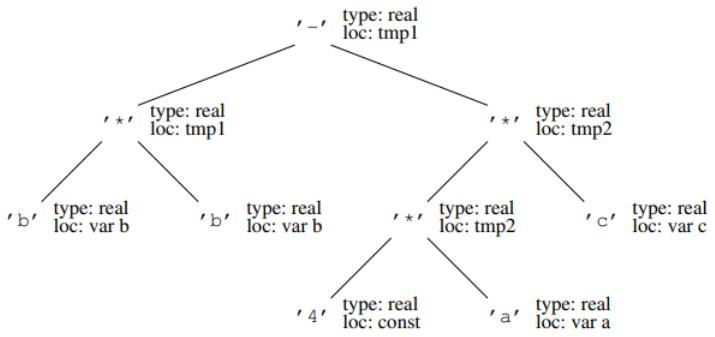
where E is an expression, T is a term, and F is a factor. The parse tree for the expression $b * b - 4 * a * c$ would look something like



Whereas the AST would be



The annotated AST would be



- **Narrow and broad compilers**: A narrow compiler reads a small part of the program, typically a few tokens, processes the information obtained, produces a few bytes of object code if appropriate, discards most of the information about these tokens, and repeats this process until the end of the program text is reached.

  A broad compiler reads the entire program and applies a series of transformations to it (lexical, syntactic, contextual, optimizing, code generating, etc.), which eventually result in the desired object code. This object code is then generally written to a file

- **N-pass compilers**: Since the "field of vision" of a narrow compiler is, well, narrow, it is possible that it cannot manage all its transformations on the fly. Such compilers then write a partially transformed version of the program to disk and, often using a different program, continue with a second pass; occasionally even more passes are used. Not surprisingly, such a compiler is called a 2-pass (or N-pass) compiler, or a 2-scan (N-scan) compiler. If a distinction between these two terms is made, "2-scan" often indicates that the second pass actually re-reads (re-scans) the original program text, the difference being that it is now armed with information extracted during the first scan.

- **Portable programs**: A program is considered portable if it takes a limited and reasonable effort to make it run on different machine types. What constitutes "a limited and reasonable effort" is, of course, a matter of opinion, but today many programs can be ported by just editing the makefile to reflect the local situation and recompiling.

- **Grammers (CFG's)**: Grammars, or more precisely context-free grammars, are the essential formalism for describing the structure of programs in a programming language. In principle the grammar of a language describes the syntactic structure only, but since the semantics of a language is defined in terms of the syntax, the grammar is also instrumental in the definition of the semantics

  There are other grammar types besides context-free grammars, but we will be mainly concerned with context-free grammars. We will also meet regular grammars, which more often go by the name of "regular expressions" and which result from a severe restriction on the context-free grammars; and attribute grammars, which are context-free grammars extended with parameters and code. Other types of grammars play only a marginal role in compiler construction. The term "contextfree" is often abbreviated to CF. We will give here a brief summary of the features of CF grammars

  A "grammar" is a recipe for constructing elements of a set of strings of symbols. When applied to programming languages, the symbols are the tokens in the language, the strings of symbols are program texts, and the set of strings of symbols is the programming language. The string

BEGIN print ("Hi!") END

consists of 6 symbols (tokens) and could be an element of the set of strings of symbols generated by a programming language grammar, or in more normal words, be a program in some programming language. This cut-and-dried view of a programming language would be useless but for the fact that the strings are constructed in a structured fashion; and to this structure semantics can be attached.

the six tokens produced by a lexical analyzer are:

- **BEGIN:** keyword
- **print:** identifier (or keyword, depending on the language specification)
- **(:** left parenthesis
- **"Hi!":** string literal
- **):** right parenthesis
- **END:** keyword

- **The form of a grammar**: A **grammar** consists of a set of production rules and a start symbol. Each production rule defines a named syntactic construct. A **production rule** consists of two parts, a left-hand side and a right-hand side, separated by a left-to-right arrow. The **left-hand side** is the name of the syntactic construct; the **right-hand side** shows a possible form of the syntactic construct. An example of a production rule is

$$\text{expression } \rightarrow' (\text{'expression operator expression})'.$$

- **Terminal and non-terminal symbols**: The right-hand side of a production rule can contain two kinds of symbols, terminal symbols and non-terminal symbols. As the word says, a **terminal symbol** (or **terminal** for short) is an end point of the production process, and can be part of the strings produced by the grammar. A **non-terminal symbol** (or **non-terminal** for short) must occur as the left-hand side (the name) of one or more production rules, and cannot be part of the strings produced by the grammar. Terminals are also called **tokens**, especially when they are part of an input to be analyzed. Non-terminals and terminals together are called **grammar symbols**. The grammar symbols in the righthand side of a rule are collectively called its **members**; when they occur as nodes in a syntax tree they are more often called its "children"
  - **Non-terminals** are denoted by capital letters, mostly A, B, C, and N.
  - **Terminals** are denoted by lower-case letters near the end of the alphabet, mostly x, y, and z.
  - **Sequences of grammar symbols** are denoted by Greek letters near the beginning of the alphabet, mostly α (alpha), β (beta), and γ (gamma).
  - **Lower-case letters near the beginning of the alphabet** (a, b, c, etc.) stand for themselves, as terminals.
  - **The empty sequence** is denoted by ε (epsilon).

- **Sentential form, production tree,and production step**: The central data structure in the production process is the sentential form. It is usually described as a string of grammar symbols, and can then be thought of as representing a partially produced program text. For our purposes, however, we want to represent the syntactic structure of the program too. The syntactic structure can be added to the flat interpretation of a sentential form as a tree positioned above the sentential form so that the leaves of the tree are the grammar symbols. This combination is also called a production tree

A string of terminals can be produced from a grammar by applying so-called production steps to a sentential form, as follows. The sentential form is initialized to a copy of the start symbol. Each production step finds a non-terminal N in the leaves of the sentential form, finds a production rule $N \to \alpha$ with N with N as its lefthand side, and replaces the N in the sentential form with a tree having N as the root and the right-hand side of the production rule, α, as the leaf or leaves. When no more non-terminals can be found in the leaves of the sentential form, the production process is finished, and the leaves form a string of terminals in accordance with the grammar.

Using the conventions described above, we can write that the production process replaces the sentential form $\beta N \gamma$ by $\beta \alpha \gamma$

- **More on sentential forms and production steps**: A production step is a single application of a grammar rule. If a grammar has a rule

$$N \to \alpha$$

then a sentential of the form

$$\beta N \gamma$$

can be written as

$$\beta \alpha \gamma.$$

  - N: A non-terminal to be expanded
  - α: The replacement string
  - β, γ: Unchanged context

When we write

$$\beta N \gamma \implies \beta \alpha \gamma$$

we are describing one production step

  - N - a non-terminal that we are going to expand
  - α - the right-hand side of a grammar rule (what replaces N)
  - β - everything to the left of N
  - γ - everything to the right of N

- **Derivation**: The steps in the production process leading from the start symbol to a string of terminals are called the derivation of that string. Suppose our grammar consists of the four numbered production rules:

  1. expression → '(' expression operator expression ')'
  2. expression → '1'
  3. operator → '+'
  4. operator → ' *'

in which the terminal symbols are surrounded by apostrophes and the non-terminals are identifiers, and suppose the start symbol is expression. Then the sequence of sentential forms shown below

```
      expression
1@1   '(' expression operator expression ')'
2@2   '(' '1' operator expression ')'
4@3   '(' '1' '*' expression ')'
1@4   '(' '1' '*' '(' expression operator expression ')' ')'
2@5   '(' '1' '*' '(' '1' operator expression ')' ')'
3@6   '(' '1' '*' '(' '1' '+' expression ')' ')'
2@7   '(' '1' '*' '(' '1' '+' '1' ')' ')'
```

Fig. 1.26: Leftmost derivation of the string (1*(1+1))

forms the derivation of the string $(1 * (1 + 1))$. More in particular, it forms a **left-most derivation**, a derivation in which it is always the leftmost non-terminal in the sentential form that is rewritten

An indication $R@P$ shows that grammar rule R is used to rewrite the non-terminal at position P. The resulting parse tree is

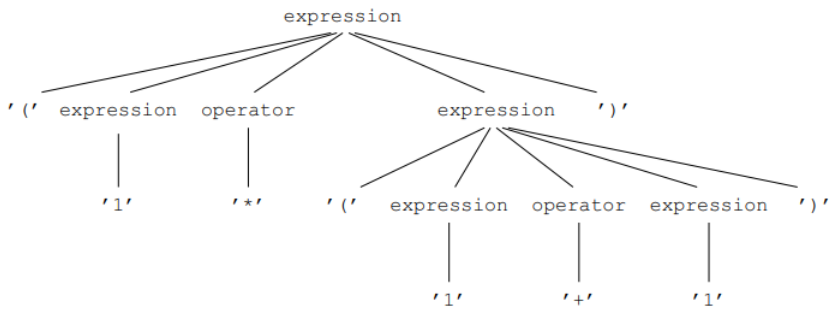


Fig. 1.27: Parse tree of the derivation in Figure 1.26

We see that recursion-the ability of a production rule to refer directly or indirectly to itself-is essential to the production process; without recursion, a grammar would produce only a finite set of strings

The production process is kind enough to produce the program text together with the production tree, but then the program text is committed to a linear medium (paper, computer file) and the production tree gets stripped off in the process. Since we need the tree to find out the semantics of the program, we use a special program, called a "parser", to retrieve it

- **Extended forms of grammars**: The single grammar rule format

$$\text{non-terminal} \rightarrow \text{zero or more grammar symbols}$$

used above is sufficient in principle to specify any grammar, but in practice a richer notation is used

The format described so far is known as BNF, which may be considered an abbreviation of Backus–Naur Form or of Backus Normal Form. It is very suitable for expressing nesting and recursion, but less convenient for expressing repetition and optionality, although it can of course express repetition through recursion. To remedy this, three additional notations are introduced, each in the form of a postfix operator:

- R^+: Indicates the occurrence of one or more Rs, to express repetition
- $R^?$: Indicates the occurrence of zero or one Rs, to express optionality
- R^*: Indicates the occurrence of zero or more Rs, to express optional repetition

Parentheses may be needed if these postfix operators are to operate on more than one grammar symbol. The grammar notation that allows the above forms is called EBNF, for Extended BNF. An example is the grammar rule

$$\text{paramater\_list} \rightarrow (\text{'IN'} \mid \text{'OUT'})^? \text{ identitfier } (\text{','} \text{ identifier})^*.$$

- **Properties of grammars**:
  - **Left-recursive non-terminal**: A non-terminal N is left-recursive if, starting with a sentential form N, we can produce another sentential form starting with N
  - **Left-recursive grammar**: By extension, a grammar that contains one or more left-recursive rules is itself called left-recursive
  - **Right-recursive**: The right version of left-recursive, not as important.
  - **Nullable non-terminal**: A non-terminal N is nullable if, starting with a sentential form N, we can produce an empty sentential form ε

    A grammar rule for a nullable non-terminal is called an ε-rule.
  - **Useless non-terminal**: A non-terminal N is useless if it can never produce a string of terminal symbols: any attempt to do so inevitably leads to a sentential that again contains N.
  - **Ambiguous grammar**: A grammar is ambiguous if it can produce two different production trees with the same leaves in the same order.

    That means that when we lose the production tree due to linearization of the program text we cannot reconstruct it unambiguously; and since the semantics derives from the production tree, we lose the semantics as well. So ambiguous grammars are to be avoided in the specification of programming languages, where attached semantics plays an important role
- **Symbols**: The basic unit in formal grammars is the symbol. The only property of these symbols is that we can take two of them and compare them to see if they are the same. In this they are comparable to the values of an enumeration type. Like these, symbols are written as identifiers, or, in mathematical texts, as single letters, possibly with subscripts. Examples of symbols are N, x, procedure_body, assignment_symbol, t_k.
- **Production rule**. Given two sets of symbols V_1 and V_2, a production rule is a pair

$$(N, \alpha) \text{ such that } N \in V_1, \alpha \in V_2^*,$$

in which X^* means a sequence of zero or more elements of the set X. This means that a production rule is a pair consisting of an N, which is an element of V_1, and a sequence α of elements of V_2. We call N the *left-hand side* and α the *right-hand side*. We do not normally write this as a pair (N, α), but rather as

$$N \rightarrow \alpha,$$

although technically it is a pair. The V in V_1 and V_2 stands for *vocabulary*.

- **CFGs**: A context-free grammar G is a 4-tuple

$$G = (V_N, V_T, S, P)$$

where V_N and V_T are sets of symbols. S is a symbol, and P is a set of production rules. The elements of V_N are the **non-terminal symbols**, and elements of V_T are the **terminal symbols**. S is called the **start symbol**.

- **Context conditions**: The previous paragraph defines only the context-free form of a grammar. To make it a real, acceptable grammar, it has to fulfill three context condition

1. $V_N \cap V_T = \varnothing$
2. $S \in V_N$
3. $P \subseteq \{(N, \alpha)\} \mid N \in V_N, \alpha \in (V_N \cup V_T)^*$

- **Strings**: Sequences of symbols are called strings.

- **Directly derivable**: A string may be derivable from another string in a grammar. More precisely, a string β is said to be *directly derivable* from a string α, written as

$$\alpha \Rightarrow \beta,$$

if and only if there exist strings δ_1, δ_2, and γ, and a non-terminal $N \in V_N$, such that

$$\alpha = \delta_1 N \delta_2, \quad \beta = \delta_1 \gamma \delta_2, \quad (N, \gamma) \in P.$$

- **Derivable**: A string β is said to be *derivable* from a string α, written as

$$\alpha \overset{*}{\Rightarrow} \beta,$$

if and only if either $\alpha = \beta$, or there exists a string γ such that

$$\alpha \overset{*}{\Rightarrow} \gamma \quad \text{and} \quad \gamma \Rightarrow \beta.$$

This means that a string is derivable from another string if we can reach the second string from the first through zero or more production steps.

- **Sentential form**: A sentential form of a grammar G is defined as

$$\alpha \mid S \overset{*}{\Rightarrow} \alpha$$

which is any string that is derivable from the start symbol S of G. Note that α may be the empty string.

- **Terminal production**: A terminal production of a grammar G is defined as a sentential form that does not contain non-terminals:

$$\alpha \mid S \overset{*}{\Rightarrow} \alpha \wedge \alpha \in V_T^*.$$

- **Language generated by a grammar** G: The language $\mathcal{L}$ generated by a grammar G is defined as

$$\mathscr{L}(G) = \{\alpha \mid S \overset{*}{\Rightarrow} \alpha \wedge \alpha \in V_T^*\}.$$

So, the set of all terminal productions of G.

If G is a grammar for a programming language, then $\mathscr{L}(G)$ is the set of all programs in that language that are correct in a context-free sense.

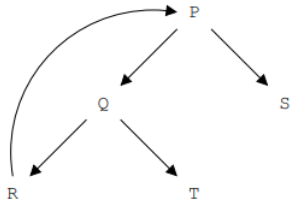
- **Sentences**: These terminal productions are called **sentences** in the language $\mathscr{L}(G)$
- **Intro to closure algorithms**: Quite a number of algorithms in compiler construction start off by collecting some basic information items and then apply a set of rules to extend the information and/or draw conclusions from them. These "information-improving" algorithms share a common structure which does not show up well when the algorithms are treated in isolation; this makes them look more different than they really are. We will therefore treat here a simple representative of this class of algorithms, the construction of the calling graph of a program, and refer back to it from the following chapters
- **Calling graph**: The calling graph of a program is a directed graph which has a node for each routine (procedure or function) in the program and an arrow from node A to node B if routine A calls routine B directly or indirectly. Such a graph is useful to find out, for example, which routines are recursive and which routines can be expanded in-line inside other routines

  The initial calling graph is, however, of little immediate use since we are mainly interested in which routine calls which other routine directly or indirectly. For example, recursion may involve call chains from A to B to C back to A. To find these additional information items, we apply the following rule to the graph: If there is an arrow from node A to node B and one from B to C, make sure there is an arrow from A to C.

  Consider the program

```
0  void P(void) { ... Q(); ... S (); ... }
1  void Q(void) { ... R(); ... T (); ... }
2  void R(void) { ... P (); }
3  void T(void) { ... }
4  void S(void) { ... }
```

  The calling graph is then

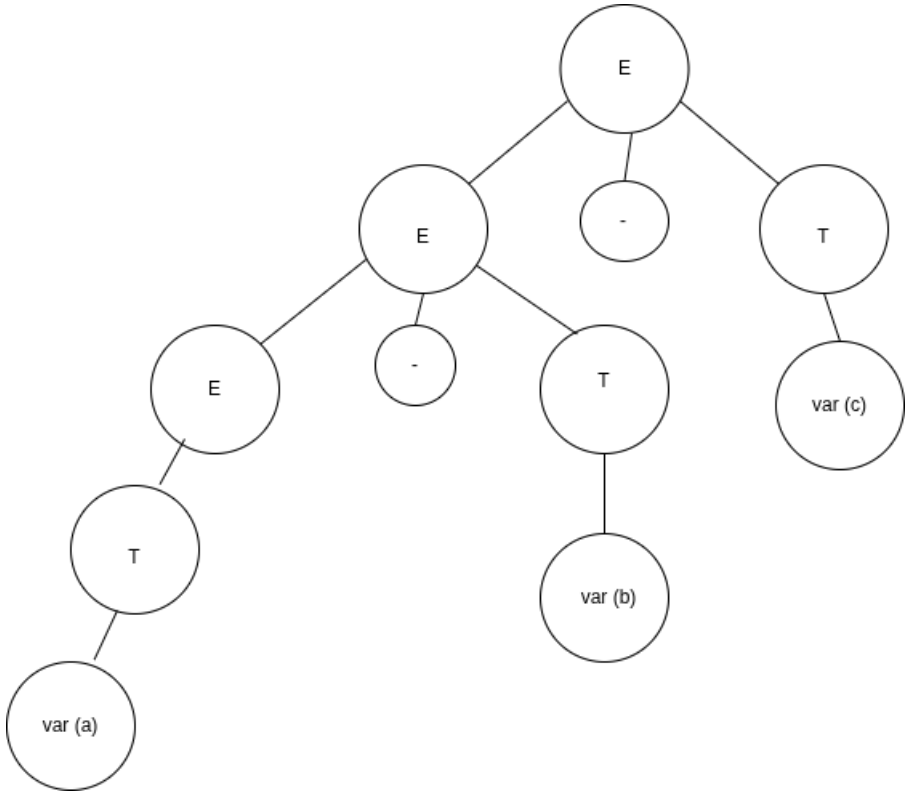


- **Transitive closure**: If we consider this rule as an algorithm (which it is not yet), this set-up computes the transitive closure of the relation "calls directly or indirectly". The transitivity axiom of the relation can be written as:

$$A \subseteq B \wedge B \subseteq C \rightarrow A \subseteq C,$$

  where the operator $\subseteq$ should be read as "calls directly or indirectly". Now, A is recursive is equivalent to $A \subseteq A$.

Adding this rule to the figure above, we get



We see that the recursion of the routines P, Q, and R has been brought into the open.

- **Components of a closure algorithm**:
  - **Data definitions**: definitions and semantics of the information items; these derive from the nature of the problem.
  - **Initializations**: one or more rules for the initialization of the information items; these convert information from the specific problem into information items.
  - **Inference rules**: one or more rules of the form: "If information items I1,I2,... are present then information item J must also be present". These rules may again refer to specific information from the problem at hand.

  The rules are called inference rules because they tell us to infer the presence of information item J from the presence of information items $I_1, I_2, ...$ When all inferences have been drawn and all inferred information items have been added, we have obtained the closure of the initial item set. If we have specified our closure algorithm correctly, the final set contains the answers we are looking for. For example, if there is an arrow from node A to node A, routine A is recursive, and otherwise it is not. Depending on circumstances, we can also check for special, exceptional, or erroneous situations

- **Recursion detection as a closure algorithm**:
  - **Data definitions:**
    1. G, a directed graph with one node for each routine. The information items are arrows in G.

2. An arrow from a node A to a node B means that routine A calls routine B directly or indirectly.

- **Initializations:** If the body of a routine A contains a call to routine B, an arrow from A to B must be present.
- **Inference rules:**: If there is an arrow from node A to node B and one from B to C, an arrow from A to C must be present.

Two things must be noted about this format. The first is that it does specify which information items must be present but it does not specify which information items must not be present; nothing in the above prevents us from adding arbitrary information items. To remedy this, we add the requirement that we do not want any information items that are not required by any of the rules: we want the smallest set of information items that fulfills the rules in the closure algorithm. This constellation is called the **least fixed point** of the closure algorithm.

The second is that the closure algorithm as introduced above is not really an algorithm in that it does not specify when and how to apply the inference rules and when to stop; it is rather a declarative,

- **Transitive closure algorithms**: General closure algorithms may have inference rules of the form "If information items $I_1, I2_2, ...$ are present then information item J must also be present", as explained above. If the inference rules are restricted to the form "If information items (A, B) and (B, C) are present then information item (A, C) must also be present", the algorithm is called a transitive closure algorithm

Lexical analysis

- **Lexical analysis**: First phase of compilation, its purpose is to convert the raw input text (source code) into a sequence of tokens.
  - Reads characters from the source program
  - Groups characters into tokens
  - Removes whitespace and comments
  - Classifies lexemes into categories such as:
    * keywords
    * identifiers
    * literals
    * operators

  Consider

  ```
  x = 10 + y;
  ```

  The tokens are

  $$\text{IDENTIFIER ASSIGN NUMBER PLUS IDENTIFIER SEMICOLON}$$

- **Syntactic analysis**: Syntactic analysis (parsing) is the second phase of compilation. Its purpose is to determine whether the token sequence follows the grammar of the language.
  - Takes tokens from the lexer
  - Checks grammatical structure
  - Builds a parse tree or syntax tree
  - Detects syntax errors

  Checks whether

  ```
  x = 10 + y
  ```

  matches the grammar rule

  $$\text{assignment} \rightarrow \text{identifier} = \text{expression};.$$

  The component that performs syntactic analysis is called the parser.

- **Interpreters and compilers**
  - **Interpreters**: Lexical analysis → parsing → execute
  - **Compilers**: Lexical analysis → parsing → code generation → executable

- **Chomsky hierarchy**: The Chomsky Hierarchy is a classification of formal grammars based on their generative power-that is, the types of languages they can describe and the computational models required to recognize them.

It consists of four levels, ordered from most restrictive to most powerful.

- **Type 3 - Regular grammars**: Of the form

$$A \to aB \quad \text{or} \quad A \to a.$$

  Recognized by finite automata (DFA / NFA). Examples are regular languages, programming language tokens, and identifiers, numbers, keywords

- **Type 2 - CFGs**: Of the form

$$A \to \alpha,$$

  where

  * A is a single non-terminal
  * α is any string of terminals and non-terminals
  * Allows recursion
  * Can represent nested structures
  * Most programming language syntax is CFG-based

  Recognized by push down automata (PDA). Examples are

  * Arithmetic expressions
  * Balanced parentheses
  * Programming language syntax

- **Type 1 - CSGs**: Of the form

$$\alpha A \beta \to \alpha \gamma \beta$$

  with

$$|\gamma| \geqslant 1.$$

  Characteristics are

  * Rules depend on surrounding context
  * Length of strings never decreases
  * More powerful than CFGs

  Recognized by Linear bounded automata (LBA)

- **Type 0 - Unrestricted grammars**: Of the form

$$\alpha \to \beta,$$

  where α contains at least one non-terminal. Characteristics are

  * No restrictions on production rules
  * Most powerful grammar type
  * Can generate all computable languages

  Recognized by turing machines. Example is an REL.

- **Regular grammars / regular languages**: A regular grammar is the most restrictive class in the Chomsky hierarchy. It generates exactly the regular languages, which are the languages recognized by finite automata.

A grammar is regular if all of its productions are of one of the following forms:

- **Right regular**:

$$A \to aB \quad \text{or} A \to a.$$

- **Left regular**:

$$A \to Ba \quad \text{or} A \to a.$$

A grammar must be either entirely right-regular or entirely left-regular - never mixed.

Consider a mixed grammar

$$S \to \varepsilon \mid aA$$
$$A \to Sb.$$

This would allow

- Non-terminals on both sides
- Multiple derivation directions
- Power beyond regular languages

Such grammars can generate non-regular languages, which breaks the definition.

Notice that the above grammar could generate the language $\mathscr{L} = \{a^n b^n \ : \ n \geqslant 0\}$, which is famously nonregular. Observe that

$$S \to \varepsilon \mid aA$$
$$A \to bS$$

yields $\mathscr{L} = \{(ab)^n \ : \ n \geqslant 0\}$, which is regular. Notice that all productions are right-linear

- **Regular languages and FAs in lexical analysis**: Lexical analysis is the first phase of compilation. Its purpose is to convert a stream of characters into a stream of tokens.

  This process is based almost entirely on finite automata. Lexical structure of programming languages is:

  - Regular
  - Pattern-based
  - Non-nested
  - Locally decidable

  All of these can be described by regular expressions, which implies regular languages, which implies FAs.

- **Token specification**: Each token class is defined using an regular expression. For example,

$$\text{identifier} \to \text{letter}(\text{letter} \mid \text{digit})^*$$
$$\text{number} \to \text{digit}^+$$
$$\text{whitespace} \to (\text{space} \mid \text{tab} \mid \text{newline})^+.$$

  Each regular expression is converted into an NFA using standard constructions like Thompson's construction with ε-transitions allowed.

Note: Note efficient to execute directly, we can instead convert the NFA to a DFA, which is possible.

- **How the DFA is used (maximal munch / longest match rule)**:
  1. Start at the initial state
  2. Read input character by character
  3. Follow transitions
  4. Track the last accepting state
  5. When no transition is possible:
     - Backtrack to last accepting state
     - Emit the corresponding token
     - Restart from next input position

Note: You use a finite automaton to extract the tokens. The FA operations on raw characters.

$$\text{Char stream} \rightarrow \text{FA} \rightarrow \text{tokens.}$$

So the FA's job is to recognize token boundaries, not to consume tokens. The outputs of the FA is the tokens.

- **Lexical analyzer example**: We will build a lexer for the following token types

$$\begin{aligned} \text{ID} &\rightarrow [a-zA-Z][a-zA-Z0-9]^* \\ \text{NUM} &\rightarrow [0-9]^+ \\ \text{PLUS} &\rightarrow + \\ \text{WS} &\rightarrow [\backslash t \setminus n]^+. \end{aligned}$$

We will use the following transition table

Current	**Letter**	**Digit**	+	**Whitespace**
q0	q1	q2	q3	q0
q1	q1	q1	-	accept
q2	-	q2	-	accept
q3	-	-	-	accept

Consider the stream

```
sum1 + 42
```

Then, reading the stream gives

Input Read	State	Action
s	q1	continue
u	q1	continue
m	q1	continue
1	q1	continue
space	-	emit ID(sum1)
+	q3	emit PLUS
space	-	ignore
4	q2	continue
2	q2	continue
EOF	-	emit NUM(42)

So, the output tokens are

```
<ID, "sum1">
<PLUS, "+">
<NUM, "42">
```

- **Invalid tokens**: A token is invalid if:
  - The input character sequence does not match any token pattern
  - The DFA reaches a state with no valid transition
  - No accepting state was reached before failure

  In this case, a lexical error is reported.
- **Where do the tokens go**: The output of the lexical analyzer goes directly to the parser. The lexer produces a stream of tokens, and these tokens are consumed one-by-one by the parser. The lexer does not store the full list of tokens permanently - it typically supplies them on demand to the parser.
- **String to int conversion**

```cpp
int string_to_int(const string& s) {
    int i = 0;
    bool negative = false;

    if (s[0] == '-') negative = true;

    for (const auto& c : s)  {
        if (isdigit(c)) { i = (i * 10) + (c - '0'); }
        else return -1;
    }

    return negative ? -i : i;
}
```

- **Hex string to int conversion**:

```cpp
 int hex_stoi(const string& s) {
    int i = 0;
    for (const auto& c : s) {
        if (isdigit(c)) {
            i = i * 16 + (c - '0');
        } else if (c >= 97 && c <= 102) {
            i = i * 16 + (c - 'a') + 10;
        } else if (c >= 65 && c <= 70) {
            i = i * 16 + ((c - 'A') + 10);
        } else return -1;
    }

    return i;
}
```

- **UTF-8**
- **Recursion in CFGs**: It is important to note that when constructing CFGs, we should use only left-recursion or only right-recursion. If a grammar is is both left and right recursive,
  - Ambiguous parse trees
  - Infinite recursion in top-down parsers
  - No fixed associativity
  - Impossible to assign precedence cleanly
  - AST construction becomes ill-defined
- **Associativity in grammars**: Consider the grammar

$$E \to E + T \mid E - T \mid T$$
$$T \to \text{var} \mid \text{int}.$$

  For an expression like $a - b - c$, there are two interpretations,
  - **Left-associative**

$$(a - b) - c.$$

– **Right-associative**

$$a - (b - c).$$

Note that most arithmetic operators are left-associative. If we notice the rule $E \to E - T$, the recursive call to E is on the left-hand side of the production. This is called left recursion. Let's derive

$$a - b - c.$$

The derivation is

$$E \implies (E - T).$$

Then, expand the left E again,

$$E - T \implies ((E - T) - T) \implies ((T - T) - T) \implies ((a - b) - c).$$

This associativity is forced by the grammar. In order to derive

$$(a - (b - c))$$

we would need the rule

$$E \to T - E.$$

Parsing

- **Parse trees (syntax trees)**: A parse tree is a tree representation of the syntactic structure of a string according to a context-free grammar.
- **Abstract syntax trees**: An Abstract Syntax Tree (AST) is a tree representation of the abstract syntactic structure of a program, where unnecessary grammatical details are removed.
- **What's the point of a parse tree**: A parse tree gives you the complete syntactic structure of an input string as dictated by a grammar. More precisely, it shows how the grammar derives the string, step by step.

  A parse tree is a concrete representation of a derivation in a context-free grammar. It tells you:

  - Which grammar rules were applied
  - In what order they were applied
  - How the input string is grouped syntactically
  - How associativity and precedence are enforced

  Every internal node corresponds to a nonterminal, every leaf corresponds to a terminal symbol.

  A parse tree proves that a string:

  - belongs to the language
  - is generated by the grammar
  - If a parse tree exists $\rightarrow$ the string is syntactically valid.
- **Parsing expressions with grammars**: In compiler theory, a grammar serves three closely related goals:

  - Define the legal syntax of expressions.
  - Guide parsing to produce a parse tree (concrete syntax tree).
  - Enable construction of an AST, which is used for semantic analysis and code generation.

  The grammar must encode:

  - Operator precedence
  - Operator associativity
  - Grouping rules (parentheses)

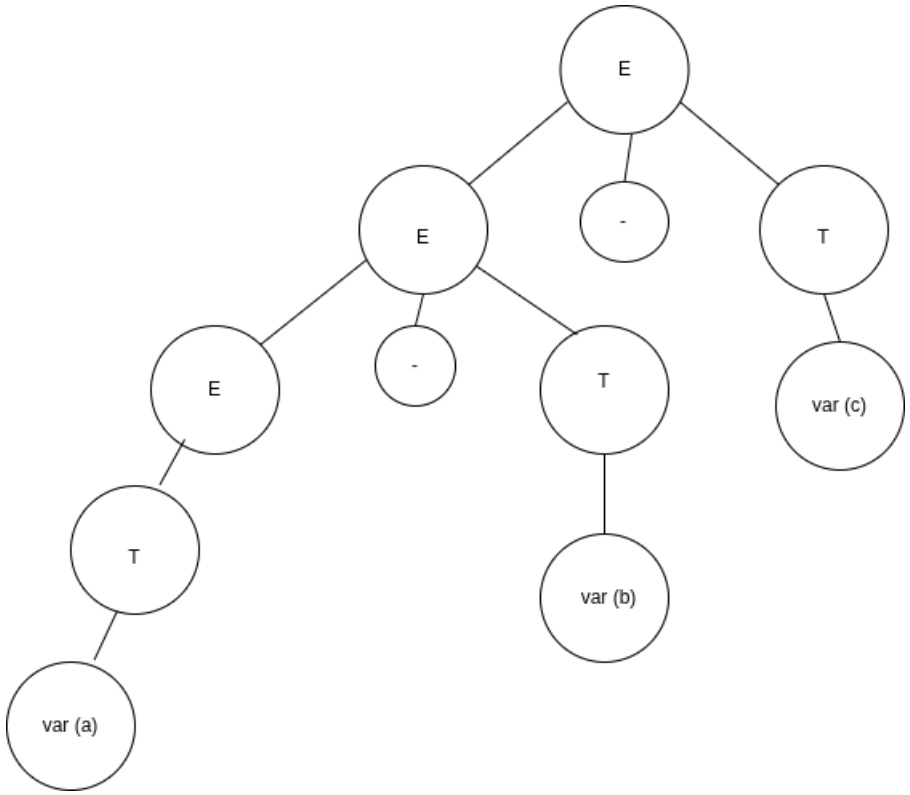
  Consider the simple grammar

  $$\begin{aligned} E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow \text{var} \mid \text{int}. \end{aligned}$$

  Consider the expression $a - b - c$, the derivation is

  $$\begin{aligned} E &\implies E - T \implies E - T - T \implies T - T - T \\ &\implies \text{var (a)} - \text{ var (b)} - \text{ var (c)}, \end{aligned}$$

  thus yielding $a - b - c$. The parse tree from this derivation is then



Note that in parse trees, the children of a node are implicitly grouped as if surrounded by parentheses, Even if no parentheses appear in the source code, the tree structure itself acts as parentheses. A parse tree represents how the grammar groups subexpressions.

- Each node = an operation
- Its children = operands
- Subtrees = grouped expressions

Note that in derivations, when we expand a non-terminal, that expansion is implicitly grouped by parenthesis, even if we do not write the parenthesis in the derivation. In fact, we shouldn't. Every application of a production rule implicitly creates a grouped subtree, the grouping exists in the tree, not in the derivation string.

Suppose that we try to extend this grammar for multiplication and division, so we might extend our grammar to

$$E \to E + T \mid E - T \mid E * T \mid E/T \mid T$$
$$T \to \text{var} \mid \text{int}.$$

The problem is that this grammar treats all operators as equal precedence. So,

$$a + b * c$$

can be derived either as $((a + b) * c)$ or $(a + (b * c))$. For the first one, the derivation is

$$E \implies (E * T) \implies ((E + T) * T) \implies ((T + T) * T) \implies ((a + b) * c).$$

The second one is derived from

$$E \implies (E + T) \implies ((E * T) + T) \implies ((T * T) + T) \implies ((b * c) + a),$$

which is the same as

$$(a + (b * c)).$$

Both are valid derivations, which yield different parse trees for the same expression. This implies an ambiguous grammar, which is unacceptable for a complier.

Compilers must:

- Produce exactly one meaning
- Generate deterministic code
- Respect mathematical precedence

Ambiguity causes:

- Undefined behavior
- Multiple possible ASTs
- Impossible semantic analysis

So we need a grammar that forces precedence structurally, not by rules outside the grammar.

The solution is to separate precedence levels into different nonterminals.

- **Structural layers**: In order to enforce precedence, we must build structural layers in our grammar, where higher precedence operators appear on lower levels. Consider
  - **E:** Expression, lowest precedence
  - **T:** Term, medium precedence
  - **F:** Factor, highest precedence

  Each layer corresponds to how tightly operators bind. Precedence is enforced by how deep an operator appears in the grammar. Lower in the grammar enforces
  - more deeply nested in the parse tree
  - evaluated earlier
  - higher precedence

  The grammar

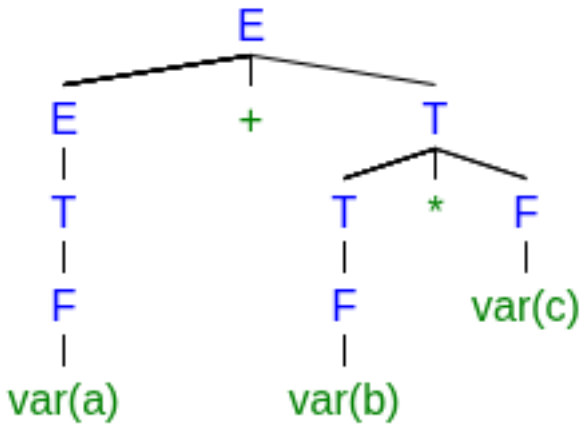
$$\begin{aligned} &E \to E + T \mid E - T \mid T \\ &T \to T * F \mid T/F \mid F \\ &F \to \text{int} \mid \text{var}. \end{aligned}$$

  Provides correct operator precedence. Consider the expression $a+b*c$. The derivation is then

$$\begin{aligned} E &\implies E + T \implies T + T \implies T + T * F \implies T + T * \text{var (c)} \\ &\implies F + F + \text{var (c)} \implies \text{var (a)} + F + \text{var (c)} \\ &\implies \text{var (a)} + \text{var (b)} + \text{var (c)}. \end{aligned}$$

  Recall that the parenthesis are not required in the derivation, as they are implicit and expressed by the parse tree. Derivations describe rule application order. Parse trees describe grouping.

The parse tree for the above derivation would be



With this knowledge its easy to see that if we switched the precedence of +,-,*,/ so that +,- appears on a deeper level than *,/, then $a+b$ would be grouped and evaluated before multiplying by c.

- **Explicit Parenthesis, custom order of operations**: We can create a new terminal (E), so our grammar would become

$$\begin{aligned} E &\to E+T \ \mid \ E-T \ \mid \ T \\ T &\to T*F \ \mid \ T/F \ \mid \ F \\ F &\to (E) \ \mid \ \text{int} \ \mid \ \text{var}. \end{aligned}$$

Note that parenthesis has highest precedence, so it must appear on the deepest level. Now, suppose we want the expression

$$(a+((b-c)*d)).$$

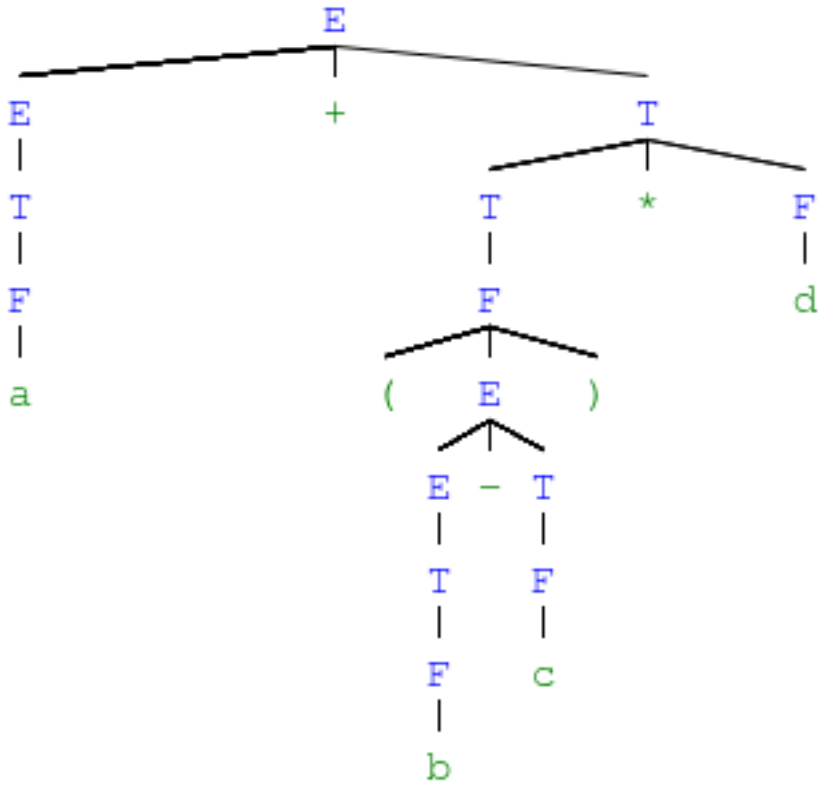
Note that most of these parenthesis are implicit i.e we do not need them, the only explicit parenthesis are

$$a+(b-c)*d.$$

However, even if parenthesis are usually implicit, but still added in the input string, we must use (E) to derive incorporate those parenthesis. For $a+(b-c)*d$ the derivation is

$$\begin{aligned} E &\implies E+T \implies E+T*F \implies E+F*F \implies E+(E)*F \\ &\implies \cdots \implies a+(E)*d \implies a+(E-T)*d \implies \cdots \implies a+(b-c)*d, \end{aligned}$$

which has parse tree



If the input string was instead

$$(a + ((b - c) * d)),$$

the derivation is

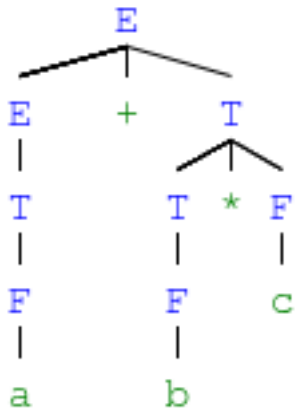
$$\begin{aligned} E &\implies T \implies F \implies (E) \implies (E + T) \implies (E + F) \implies (E + (E)) \\ &\implies (E + (T)) \implies (E + (T * F)) \implies (E + (F * F)) \implies (E + ((E) * F)) \\ &\implies (E + ((E - T) * F)) \implies \cdots \implies (a + ((b - c) * d)). \end{aligned}$$

- **AST's, parse tree to AST**: Turning a parse tree into an Abstract Syntax Tree (AST) is one of the most important conceptual steps in a compiler. The key idea is: A parse tree represents syntax. An AST represents meaning.

  A parse tree includes

  - Every nonterminal (E, T, F)
  - Every production rule used
  - Parentheses as grammar artifacts
  - Structural nodes that carry no semantic meaning

  The parse tree for $a + b * c$ is



This tree

- Is correct
- Is verbose
- Contains nodes that exist only to enforce grammar rules

But a compiler does not need most of this information. An AST

- Removes grammar-specific nodes
- Keeps only semantic structure
- Represents computation directly
- Is independent of parsing strategy

For $a + b * c$, the AST is simply

If a node exists only to enforce grammar structure, remove it. If a node represents an operation or value, keep it.

To convert an parse tree to AST, we follow the following steps

1. **Remove Non-semantic Nodes**: Nodes like E, T, F, parenthesis, and single child chains are removed.

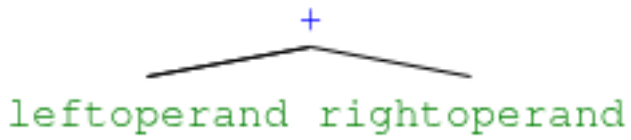
   $$E \to T \to F \to a$$

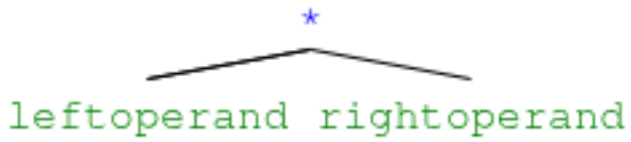
   becomes simply a.

2. **Promote operators**: Nodes like

$$E \to E + T,$$
$$T \to T * F$$

become





3. **Preserve hierarchy**: Because the grammar enforced precedence, the tree already has correct nesting, no additional rules are needed.

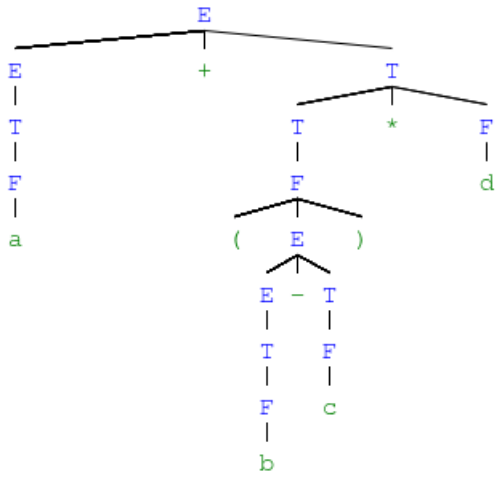
A parse tree answers "how was this derived?" An AST answers "what does this compute?"

In essence, to build an AST, remove grammar-only nodes from the parse tree and retain only operators and operands arranged according to the tree's structure.

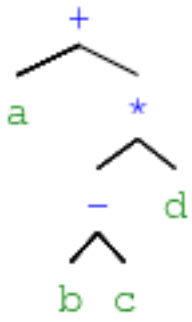
Algorithmically,

1. Visit parse tree node
2. If node is:
   - Operator → create AST node
   - Literal → create leaf
   - Grammar-only node → skip
3. Recursively process children
4. Return AST node upward

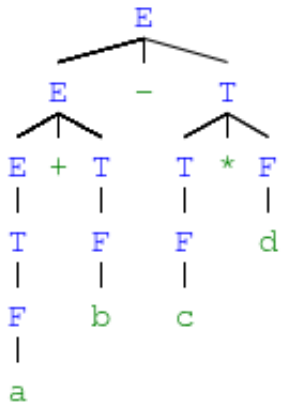
Notice that the parse tree



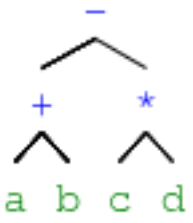
becomes the AST



If the expression $a + b - c * d$ had no explicit parenthesis, the parse tree would be



So, the AST would be



- **Evaluating ASTs**: A post-order traversal of an expression AST produces Reverse Polish Notation (RPN).

  This is not a coincidence; it is a direct consequence of how ASTs represent computation.

  In an AST:

  - Internal nodes = operators
  - Leaves = operands
  - Children = arguments to the operator

  Post-order traversal visits:

  - Left subtree
  - Right subtree
  - Node itself

  This is precisely

  operand operand operator

  which is reverse polish notation.

- **Negation and exponentiation**: To introduce unary negation and exponentiation correctly, we must extend the grammar without breaking precedence or associativity. This requires adding one new level and being careful about recursion direction. Negation binds tighter than multiplication or division, but looser than exponentiation. Exponentiation binds looser than parenthesis. So, the grammar becomes

  $$E \to E + T \mid E - T \mid T$$
  $$T \to T * U \mid T/U \mid U$$
  $$U \to -U \mid P$$
  $$P \to F \wedge P \mid F$$
  $$F \to (E) \mid \text{int} \mid \text{var}.$$

  Note that exponentiation is right-associative, so right recursion works here.

  $$a \wedge b \wedge c = a \wedge (b \wedge c).$$

3.1 Parsing algorithms

- **LL and LR families of deterministic parsers for CFGs**: The LL and LR families are the two major classes of deterministic parsers for context-free grammars. They differ in direction of parsing, derivation strategy, grammar requirements, and implementation style.
  - **LL** = Left-to-right scan, Leftmost derivation
  - **LR** = Left-to-right scan, Rightmost derivation (in reverse)

  Both read the input left-to-right. What differs is how they construct the parse.

- **The LL family (top-down parsers)**:
  - **First L:** scan input Left to right
  - **Second L:** produce a Leftmost derivation

  They start from the start symbol and try to expand it to match the input. They:
  - Maintain a stack of grammar symbols (or recursive call stack)
  - Look at the next input token
  - Choose a production for the top nonterminal
  - Expand until terminals match the input

  This corresponds exactly to recursive-descent parsing.

  Note: LL parsers **cannot** handle left recursion.

- **LL(k)**: Uses k tokens of lookahead. So, $LL(1)$ uses only one token, very common.

- **The LR family**:
  - **First L:** scan input Left to right
  - **R:** construct a Rightmost derivation in reverse

  They start from the input and reduce it back to the start symbol. These parsers are **bottom up**

- **Left-recursion elimination**: For a rule of the form

$$A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid \beta$$

  where each β does not start with A, we transform to

$$\begin{aligned} A &\to \beta A' \\ A' &\to \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \varepsilon. \end{aligned}$$

  Notice that expansions of A before elimination of left-recursion take the form $\beta(\alpha_1 + \alpha_2 + ... + \alpha_k)^*$. In the updated grammar, the first production rule produces

$$\beta A'.$$

  Now, notice that A' has expansions of the form $(\alpha_1 + \alpha_2 + ... + \alpha_k)^*$. So, the updated grammar produces

$$\beta(\alpha_1 + \alpha_2 + \cdots + \alpha_k)^*.$$

Since the two grammars produce the same derivations, they are equivalent. Consider the first rule in our left-recursive grammar

$$E \to E + T \ \mid \ E - T \ \mid \ T.$$

Notice that $\beta = T$, $\alpha_1 = +T$, and $\alpha_2 = -T$. Thus, the production rule becomes

$$\begin{aligned} E &\to TE' \\ E' &\to +TE' \ \mid \ -TE' \ \mid \ \varepsilon. \end{aligned}$$

Notice that they both produce derivations of the form

$$T(+T \ \mid \ -T)^*.$$

The full grammar would be converted to

$$\begin{aligned} E &\to TE' \\ E' &\to +TE' \ \mid \ -TE' \ \mid \ \varepsilon \\ T &\to NT' \\ T' &\to *NT' \ \mid \ /NT' \ \mid \ \text{MOD } NT' \ \mid \ \varepsilon \\ N &\to +N \ \mid \ -N \ \mid \ F \\ F &\to SF' \\ F' &\to \wedge SF' \ \mid \ \varepsilon \\ S &\to (E) \ \mid \ \text{int} \ \mid \ \text{var}. \end{aligned}$$

Notice that the rules that are not left recursive can remain the same.

- **LL(1) grammars**: A context-free grammar is called LL(1) if it can be parsed by a deterministic LL(1) parser. That is, using left-to-right scanning, leftmost derivation, and one token of lookahead with no backtracking. Thus, LL(1) is a subclass of CFGs.

  If a CFG is LL(1), there exists a deterministic predictive parser that, at every step, can choose the correct production using only the next input symbol. No guessing. No backtracking. No multiple possibilities.

- **FIRST / FOLLOW sets**:

  - **FIRST set**: The FIRST set of a non-terminal contains all the terminal symbols that can appear at the beginning of any string derived from that non-terminal (starting at that nonterminal). In other words, it tells us which terminal symbols are possible when expanding a non-terminal.

    $$\text{FIRST}(X) = \{a \ \mid \ X \stackrel{*}{\Rightarrow} a\alpha, a \text{ terminal}\}.$$

    Also includes ε if X can derive the empty string. Also, terminals derive themselves, so

    $$\text{FIRST}(a) = \{a\}$$

    for any terminal a, and

    $$\text{FIRST}(\varepsilon) = \{\epsilon\}.$$

    For each production

    $$A \to \alpha,$$

    add FIRST(α) to FIRST(A). For a string of symbols

    $$\alpha = X_1 X_2 \cdots X_k,$$

    compute FIRST(α) as follows,

1. Add FIRST(X_1) - $\{\varepsilon\}$
2. If $\varepsilon \in$ FIRST(X_1), and FIRST(X_2) $- \{\epsilon\}$
3. Continue left to right
4. If all X_i can derive ε, then include ϵ in FIRST(α)

– **FOLLOW set**: The FOLLOW set of a non-terminal contains all the terminal symbols that can appear immediately after that non-terminal in any derivation. It helps identify what can follow a non-terminal in the grammar and is essential for handling productions where a non-terminal is at the end of a rule.

$$\text{FOLLOW}(A) = \{a \mid S \stackrel{*}{\Rightarrow} \alpha A a \beta\}.$$

First, we place end marker in FOLLOW of start symbol

$$\$ \in \text{FOLLOW}(S).$$

For any production

$$A \to \alpha B a \beta,$$

where a is a terminal,

$$a \in \text{FOLLOW}(B).$$

For any production

$$A \to \alpha B \beta,$$

where $\beta \neq \varepsilon$, add

$$\text{FIRST}(\beta) - \{\varepsilon\}$$

to FOLLOW(B). If $\varepsilon \in$ FIRST(β), add FOLLOW(A) to FOLLOW(B). If FIRST(β) is empty, then add FOLLOW(A) to FOLLOW(B), so that

$$\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$$

For example, consider the grammar

$$S \to AB$$
$$A \to a$$
$$B \to b.$$

Terminals derive themselves, so

$$\text{FIRST}(a) = \{a\}$$
$$\text{FIRST}(b) = \{b\}.$$

Now nonterminals,

$$A \to a \implies \text{FIRST}(A) = \{a\}$$
$$B \to b \implies \text{FIRST}(B) = \{b\}.$$

For $S \to AB$, since A cannot derive ε,

$$\text{FIRST}(S) = \text{FIRST}(A) = \{a\}.$$

For the FOLLOW sets, put \$ in FOLLOW of start symbol, use occurrences on right-hand sides.

$$\text{FOLLOW}(S) = \{\$\}.$$

- **What is the purpose of $**: The special marker $ marks the **end of input**. It lets the parser detect when the entire input has been successfully consumed. Without an explicit end marker, the parser cannot distinguish between:
  - A complete valid parse
  - A valid prefix followed by garbage
  - Premature termination

  Regarding the start symbol S,

  $$\$ \in \text{FOLLOW}(S).$$

  Means the start symbol may be followed by end-of-input.

- ε **in FIRST and FOLLOW sets**: It is important to note that ε is not a terminal, and since FOLLOW sets contain only terminals, it therefore cannot appear. However, we let ε appear in FIRST sets to indicate if a nonterminal is nullable, and is therefore necessary in building other FIRST sets.

- **FIRST of a sequence**: Consider the rule

  $$E \to TE'.$$

  Here, $\alpha = TE'$, so we need $\text{FIRST}(TE')$. But, this may require $\text{FIRST}(T)$ and $\text{FIRST}(E')$

  For a sequence $X_1 X_2 \dots X_n$:

  1. Add $\text{FIRST}(X_1) \setminus \{\varepsilon\}$
  2. If $X_1 \Rightarrow^* \varepsilon$, also consider X_2
  3. Continue until a symbol cannot derive ε
  4. If all symbols can derive ε, include ε

  Recall that a non-terminal is **nullable** if it can derive ε. So, if T is **not** nullable,

  $$\text{FIRST}(TE') = \text{FIRST}(T).$$

  If T is nullable,

  $$\text{FIRST}(TE') = (\text{FIRST}(T) \setminus \{\varepsilon\}) \cup \text{FIRST}(E'),$$

  and ε is included only if both T and E' are nullable.

- **Showing that a grammar is LL(1)**: There are two standard methods used in compiler theory. The first is the FIRST / FOLLOW disjointness conditions A grammar is LL(1) if and only if for every nonterminal A with multiple productions.

  $$A \to \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n,$$

  the following conditions hold.

  1. **FIRST sets must be pairwise disjoint**: For all $i \neq j$,

     $$\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \varnothing.$$

     This ensures that the parser can choose a production based solely on the next input symbol.

FIRST sets must be pairwise disjoint because an LL(1) parser must choose exactly one production using only one lookahead token. If two alternatives can begin with the same terminal, the parser cannot decide which production to apply without additional lookahead or backtracking - both forbidden in LL(1).

Consider a nonterminal

$$A \to \alpha_1 \mid \alpha_2.$$

An LL(1) predictive parser decides which rule to use by inspecting the next input symbol a.

$$a \in \text{ FIRST}(\alpha_i) \implies \text{ choose } \alpha_i.$$

For this decision to be unique

$$\text{FIRST}(\alpha_1) \cap \text{ FIRST}(\alpha_2) = \varnothing.$$

2. **If ε is possible, FIRST and FOLLOW must be disjoint**: If some $\alpha_i \overset{*}{\Rightarrow} \epsilon$, then for every $j \neq i$,

$$\text{FIRST}(\alpha_j) \cap \text{FOLLOW}(A) = \varnothing.$$

If A can disappear, the parser must decide between expanding A or skipping it based on the lookahead - which must not conflict with any other production. When the parser encounters A, it must decide between

(a) Expanding A using a non-ε production, or
(b) Letting $A \implies \varepsilon$ (produce nothing)

But, producing nothing means the parser will next see whatever terminal can legally follow A in the surrounding context, precisely the terminals in FOLLOW(A). Thus, the parser uses

- FIRST sets to choose real expansions
- FOLLOW set to decide when to use ε

If these sets overlap, the parser cannot decide.

- If the symbol belongs to FIRST of some real alternative A expand
- If the symbol belongs to FOLLOW(A) $\to$ choose ε

- **When is a grammar not LL(1)**:
  - **Immediate left recursion**

$$A \to A\alpha \mid \beta.$$

  - **Indirect left recursion**

$$A \overset{+}{\Rightarrow} A\alpha.$$

  - **Ambiguous grammar**

- **Note on nullability**: A nonterminal X is nullable if and only if its FIRST set contains the empty string ε.

$$X \overset{*}{\Rightarrow} \varepsilon \iff \varepsilon \in \text{FIRST}(X).$$

The same applies for a RHS sequence α.

- **FIRST / FOLLOW sets for our LL(1) grammar**: Recall the grammar

$$\begin{aligned}
E &\to TE' \\
E' &\to +TE' \mid -TE' \mid \varepsilon \\
T &\to NT' \\
T' &\to *NT' \mid /NT' \mid \text{MOD } NT' \mid \varepsilon \\
N &\to +N \mid -N \mid F \\
F &\to SF' \\
F' &\to \wedge SF' \mid \varepsilon \\
S &\to (E) \mid \text{int} \mid \text{var}.
\end{aligned}$$

Note that the terminals are

$$\{+, -, *, /, \text{ MOD }, \wedge, (,), \text{int}, \text{var}\}.$$

So, we find the FIRST sets of all non-terminals on the right hand side of our production rules. Then, any sequences α can be derived from those sets. Recall that the FIRST set of a non-terminal X is found by determining the nonterminals that can begin a string derived starting at that nonterminal. So, those sets are

$$\begin{aligned}
\text{FIRST}(S) &= \{(, \text{int}, \text{var}\} \\
\text{FIRST}(F') &= \{\wedge, \varepsilon\} \\
\text{FIRST}(F) &= \text{FIRST}(S) = \{(, \text{int}, \text{var}\} \\
\text{FIRST}(N) &= \{+, -\} \cup \text{FIRST}(F) = \text{FIRST}(S) = \{+, -, (, \text{int}, \text{var}\} \\
\text{FIRST}(T') &= \{*, /, \text{MOD}, \varepsilon\} \\
\text{FIRST}(T) &= \text{FIRST}(N) = \{+, -, (, \text{int}, \text{var}\} \\
\text{FIRST}(E') &= \{+, -, \epsilon\} \\
\text{FIRST}(E) &= \text{FIRST}(T) = \{+, -, (, \text{int}, \text{var}\}.
\end{aligned}$$

Note the order in which we found these sets. For the FOLLOW sets, we start by initializing the FOLLOW set of the start symbol E,

$$\text{FOLLOW}(E) = \{\$\}.$$

Recall that the FOLLOW set for a nonterminal contains all the terminals that can appear immediately after that nonterminal in some sentential form derived from the start symbol.

$$a \in \text{FOLLOW}(A) \iff S \stackrel{*}{\Rightarrow} \alpha A a \beta.$$

These sets are therefore

$$\begin{aligned}
\text{FOLLOW}(E) &= \{\$,)\} \\
\text{FOLLOW}(E') &= \text{FOLLOW}(E) = \{\$,)\} \\
\text{FOLLOW}(T) &= \text{FIRST}(E') \setminus \{\varepsilon\} \cup \text{FOLLOW}(E) = \{+, -, \$,)\} \\
\text{FOLLOW}(T') &= \text{FOLLOW}(T) = \{+, -, \$,)\} \\
\text{FOLLOW}(N) &= \text{FIRST}(T') \setminus \{\varepsilon\} \cup \text{FOLLOW}(T') = \{*, /, \text{MOD}, +, -, \$,)\} \\
\text{FOLLOW}(F) &= \text{FOLLOW}(N) = \{*, /, \text{MOD}, +, -, \$,)\} \\
\text{FOLLOW}(F') &= \text{FOLLOW}(F) = \{*, /, \text{MOD}, +, -, \$,)\} \\
\text{FOLLOW}(S) &= \text{FIRST}(F') \setminus \{\varepsilon\} \cup \text{FOLLOW}(F) = \{\wedge, *, /, \text{MOD}, +, -, \$,)\}
\end{aligned}$$

.

Then, the FIRST sets for all sequences are

$$\begin{aligned}
\text{FIRST}(\varepsilon) &= \{\varepsilon\} \\
\text{FIRST}(TE') &= \{+, -, (, \text{int}, \text{var}\} \\
\text{FIRST}(+TE') &= \{+\} \\
\text{FIRST}(-TE') &= \{-\} \\
\text{FIRST}(NT') &= \{+, -, (, \text{int}, \text{var}\} \\
\text{FIRST}(*NT') &= \{*\} \\
\text{FIRST}(/NT') &= \{/\} \\
\text{FIRST}(\text{MOD}NT') &= \{\text{MOD}\} \\
\text{FIRST}(+N) &= \{+\} \\
\text{FIRST}(-N) &= \{-\} \\
\text{FIRST}(SF') &= \{(, \text{int}, \text{var}\} \\
\text{FIRST}(\wedge SF') &= \{\wedge\} \\
\text{FIRST}((E)) &= \{(\} \\
\text{FIRST}(\text{int}) &= \{\text{int}\} \\
\text{FIRST}(\text{var}) &= \{\text{var}\}.
\end{aligned}$$

- **Basics on parsing (arithmetic expressions)**: For arithmetic expressions, you typically structure the grammar to encode precedence and associativity, so the parser produces the "right" AST shape.

  Note that we use a lexer to first get all tokens, then consume the token sequence to build the AST

  For a parser whose language is arithmetic expressions, you should accept exactly the token kinds that can legally appear in that language - and reject everything else as a syntax error.

  A typical expression language needs:

  - **Numeric literals**: Integers and/or floats
  - **Arithmetic operators**: $-$ $+$ $-$ $*$ $/$ (optionally %, $\wedge$, etc.)
  - **Parentheses**: ( )
  - **Identifiers**: variable names (e.g., x, sum, foo)
  - **EOF**: end-of-input marker (very important)

  A string literal (e.g., "hello") is not part of arithmetic expressions in the usual sense. It is important that we do not silently discard it, we instead must treat it as a syntax error.

  Consider a grammar that accepts arithmetic expressions,

$$\begin{aligned}
&E \to E + T \mid E - T \mid T \\
&T \to T * N \mid T/N \mid T \text{ MOD } N \mid N \\
&N \to +N \mid -N \mid F \\
&F \to F \wedge S \mid S \\
&S \to (E) \mid \text{int} \mid \text{var}.
\end{aligned}$$

  Thus, the following operators are acceptable

- Addition, subtraction
- Multiplication, division, modulus
- Unary negation, unary plus
- Exponentiation

- **LL(1) Recursive descent without a decision matrix**: We already know that a right-recursive language is suitable for recursive descent, and that language is precisely given by the grammar

$$
\begin{aligned}
E &\to TE' \\
E' &\to +TE' \mid -TE' \mid \varepsilon \\
T &\to NT' \\
T' &\to *NT' \mid /NT' \mid \text{MOD } NT' \mid \varepsilon \\
N &\to +N \mid -N \mid F \\
F &\to SF' \\
F' &\to \wedge SF' \mid \varepsilon \\
S &\to (E) \mid \text{int} \mid \text{var}.
\end{aligned}
$$

  An LL(1) parser uses

  - **Input buffer**: The token stream
  - **Lookahead mechanism**
  - **The implicit call stack**
  - **Optional structures for building the AST**

  If you want a tree:

  - Nodes allocated dynamically
  - Returned from functions
  - Or constructed incrementally

  Without this, the parser merely validates syntax.

  Note: The lookahead token is the **next unconsumed token**, is it the only *"current token"* we need to track.

  The selection rule for LL(1) given the lookahead token a is

  1. If $a \in \text{FIRST}(\alpha_i)$, choose the production α_i, otherwise
  2. If $\varepsilon \in \text{FIRST}(\alpha_k)$, and $a \in \text{FOLLOW}(A)$, choose α_k,
  3. Otherwise, syntax error.

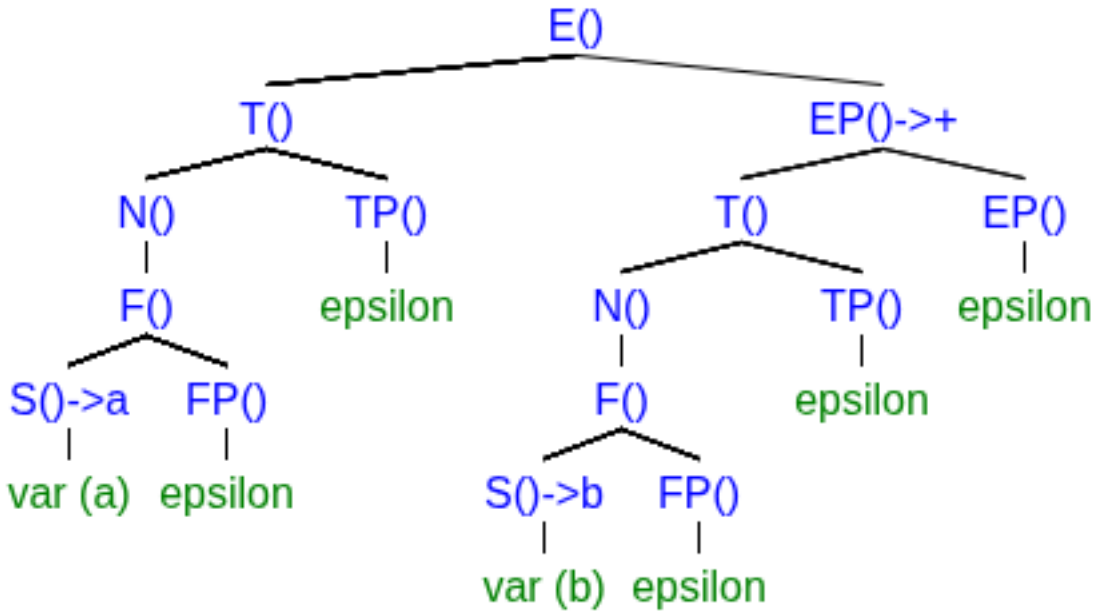
  Note that

$$\varepsilon \in \text{FIRST}(\alpha_k) \iff a_k \Rightarrow^* \varepsilon.$$

  So, given our current lookahead token, we choose a production from the selection rules above. But, does this action consume the token? No. A token is consumed only if the top of the stack contains only a terminal that matches the token. For example, if our current token is a, with token type `var`, and we choose a production $S \to \text{var}$. Then, we consume the token. This consumption should then be visible to all other calls in the call stack. In other words, the lookahead should be global, not local to any particular rule.

In the case of actually building the AST, we should return a node if we consumed a token.

Consider the expression $a + b$, the parse tree is generated implicitly by the call tree, and the AST is generated explicitly through returned nodes.



- **LL(1) parse tables**: A parsing table (specifically for LL(1) parsing) is a deterministic decision structure that tells the parser which production rule to apply based on
  - The current nonterminal to be expaneded
  - The next input token (lookahead)

  It eliminates backtracking by precomputing all valid choices.

  An LL(1) table is a 2-dimensional matrix:

  $$M[A, a]$$

  where
  - A = nonterminal (row)
  - a = terminal or \$ (column)
  - Entry = production $A \to \alpha$

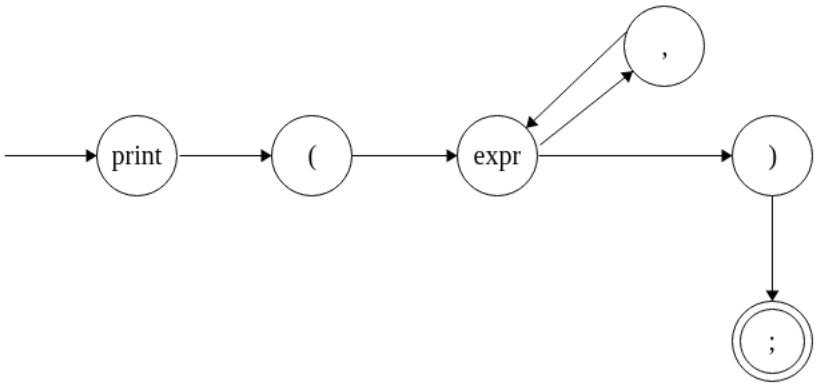
3.2 More parsing and semantics

- **Syntax diagrams**: A syntax diagram (also called a railroad diagram) is a graphical representation of a context-free grammar production used to describe the syntax of a programming language.

  More formally, a syntax diagram represents the set of valid strings generated by a grammar rule as a directed graph where paths correspond to valid derivations. Consider a print statement

  ```
  print(expr1, expr2,...);
  ```

  Statements of this form can be expressed by the syntax diagram



  Note: Syntax diagrams are closely related to parsing, but they are not themselves part of the parsing process. Instead, they are a method for specifying the syntax of a language, which a parser then implements.

- **Code diagrams**: Code diagrams are comprised of **code blocks**, which are a essentially a black box for certain operations. Between these code blocks is the actual code that we are concerned with.

- **Statement blocks and their AST representation**: In compiler theory, a statement block is a syntactic construct that groups multiple statements so they can be treated as a single statement within the language grammar and semantics.

A statement block is a sequence of statements enclosed by delimiters (such as {} in C-like languages) that forms a single compound statement. Formally, a grammar rule might look like

$$B \to \{S_1\ S_2 \dots S_n\}.$$

The purpose of blocks is to:

- group multiple statements into one unit
- define scope for variables
- serve as the body of constructs such as if, while, and for.

In an **Abstract Syntax Tree**, a statement block is typically represented by a block node whose children are the statements contained in the block.

- **Semantic analysis**: Semantic analysis is the compiler phase that checks whether a program is meaningful according to the language's semantic rules, after the program has already been proven syntactically correct by the parser.

  Formally, semantic analysis verifies that the structure produced by the parser (usually an AST) obeys the context-sensitive constraints of the language.

  These constraints cannot generally be enforced by a context-free grammar, so they must be checked after parsing. The typical compiler pipeline is:

  1. Lexical analysis → tokens
  2. Syntax analysis (parsing) → parse tree / AST
  3. Semantic analysis → validated AST with type information
  4. Intermediate representation generation
  5. Optimization
  6. Code generation

  The main responsibilities of a semantic analyzer are

  1. **Name Resolution (Identifier Binding)**: The compiler verifies that identifiers refer to valid declarations. In

     ```
     x = y + 1
     ```

     If y has not been declared, the semantilyzer reports an error. This requires symbol tables and scope management.
  2. **Type checking**: The compiler ensures that operations are applied to compatible types.

     ```
     int x;
     x = "hello";
     ```

This could be syntactically valid but semantically incorrect.

3. **Type inference and annotation**: The analyzer often annotates the AST with type information.
4. **Scope and Visibility Rules**: Semantic analysis enforces rules such as:
   - variable scope
   - shadowing
   - function visibility
   - block scope
5. **Additional Language Constraints**: Things like
   - correct number of function arguments
   - return statements matching function type
   - break/continue only inside loops
   - duplicate declarations

Context free grammars cannot enforce such constraints.

- **Working with strings**: While doing code generation in C / C++, we can set aside memory for our compiled program by simply creating a buffer. So, our "initialized data" memory may simply be a char buffer. Then, to deal with strings, we can simply place the contiguous char bytes in this memory.

  Note that we either need to add a null terminator like character to the end of each string, or track the number of bytes in each string.

  Once our strings are in the buffer (our memory), we can use the idea presented above, using high level C / C++ functions to work with those strings, for example output them, since the address of each string is the address of the first byte of the string in our buffer.

- **Variables / symbol table**: To deal with variables, we must create a symbol table. This is a structure that has information about each variable in the program. For each symbol, have

  1. Name,
  2. Type,
  3. Location

  Note that we say *location* here instead of *address* because the location may be referring to a register.

  Note about interpreters: In interpreters, variables are structures whose type is determined / can be changed dynamically at runtime, and interpreters may not make use of a symbol tables. To change information about a variable at runtime, for example the type, we simply need to change the type property of the structure that defines it.

- **Const**: Note that const is something enforced at the symbol table level. As far as the CPU and x86-64 is concerned, memory is read-write, nothing is constant. It is the job of the compiler to enforce what can / cannot be changed.

- **Variables declarations**: For the purposes of our compiler, we introduce a type `int4`, a four byte integer. Then, we may see variable declarations of the form

- `int4 var_name;`

When we see statements of this form, we place the variable into our symbol table with its name, type, and location. Regarding the location, we could just one char buffer for all needed memory in the program. Or, we could make another integer buffer for memory that is strictly four bytes. This would ensure that everything in this buffer is aligned on a four byte boundary (doubleword boundary in IA-32e), which makes things efficient for the CPU using IA-32e.

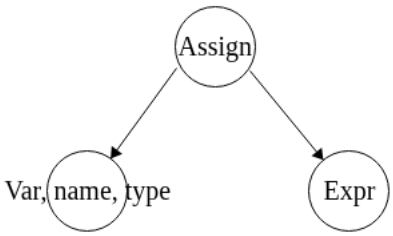
If we instead opt to use the char buffer, we could instead pad memory so that everything is placed on a doubleword (4 byte) boundary.

- **Assignments**: Consider an assignment statement

  - `var = expr;`

Recall that `var` will have token type *identifier*. Thus, since keywords share this same type, we must exhaust checks for valid keywords before we can confirm that the identifier is a variable name. If the name is not found in the symbol table, we have an error. Thus, variables must be declared before used. For functions, may need to forward declare.

The syntax tree for this statement is



3.2.1 Expanding the parser

- **Arithmetic, relational, and logical expressions**: Up to this point our study of parsing was mainly limited to arithmetic expressions. However, we also need to consider relational and logical expressions as well.

  Note that the precedence from highest to lowest is generally

  1. Arithmetic
     - $\neg$
     - $\wedge$
     - $*, /, \%$
     - $+, -$
  2. Relational
     - $<, \leqslant, >, \geqslant$
     - $==, != $
  3. Logical
     - $\sim$ (not)
     - &&
     - ||

  We shall have three sections in our parser. One section that parses logical, one section that parses relational, and one section that parses arithmetic.

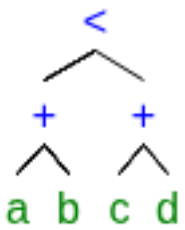
  The top level is the logical parser, the middle level is relational, and the bottom level is arithmetic. Consider an expression

  Consider an arbitrary expression. It could be logical, relation, arithmetic, or some mix. For example, consider

  $$a + b < c + d.$$

  Our parser will descend down to the arithmetic section and parse $a + b$. At this point the next token is relation, so we float back up to the relational section to consume the relational operator $<$. Then, the parser descends back down to the arithmetic section to parse $c + d$.

  At the end, we will have



This layered structure is precisely how we maintain precedence between operator classes.

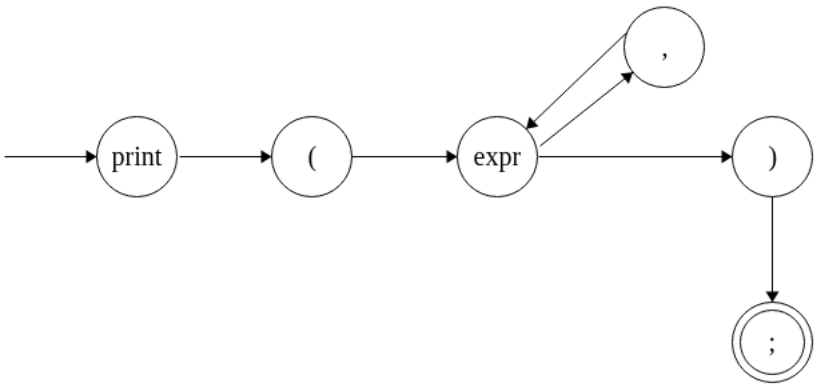
- **Print statements**: Now, consider a statement

```
print(4<7, 4+7, "hello");
```

Note that in our context, `print` will of token type *identifier*. The topmost layer of our parser will check that if we have an identifier, see if its a reserved word, and specifically which reserved word it is. If it's print, we may have something like

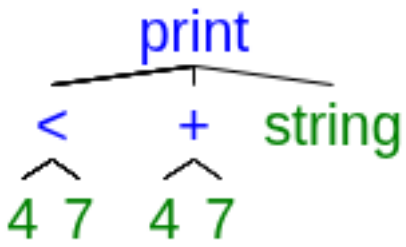
```
if (token.id == TOKEN_IDENTIFIER) {
    if (token.identifier == "print") parse_print();
    else parse_identifier();
}
...
```

Then, the section that parses print statements will further make calls to other sections of the parser to parse its arguments. Recall that the syntax diagram for a print statement is



Which we can simulate with standard control flow. Once a left parenthesis or comma is consumed, we let the part of the parser that deals with operators and expressions run. At the end of each expression parse, we then check if the next token is a right parenthesis, and in the case of our toy language, a semicolon. Only then can we say that the syntax is valid.

The AST for the expression `print(4<7, 4+7, "hello");` would be



- **Defining a boolean**: Recall that for our language, we have no boolean type, but we have the type `int4`. Thus, we will use this type to handle booleans. For our purposes, we will say that one is true and zero is false. So, we only care about the first byte, and in that first byte, the first bit.
- **Syntax diagrams and grammar for arithmetic, relational, and logical expressions**: First, let's come up with a simple grammar that handles these types of expressions, with correct precedence and associativity.

$$
\begin{aligned}
A &\to A \text{ or } B \mid B \\
B &\to B \text{ and } C \mid C \\
C &\to \sim C \mid D \\
D &\to F == F \mid F \neq F \mid E \\
E &\to F < F \mid F \leqslant F \mid F > F \mid F \geqslant F \mid F \\
F &\to F + G \mid F - G \mid G \\
G &\to G * H \mid G/H \mid G \text{ mod } H \mid H \\
H &\to \neg I \mid \oplus I \mid I \\
I &\to I \wedge J \mid J \\
J &\to (A) \mid \text{int} \mid \text{ true } \mid \text{ false } .
\end{aligned}
$$

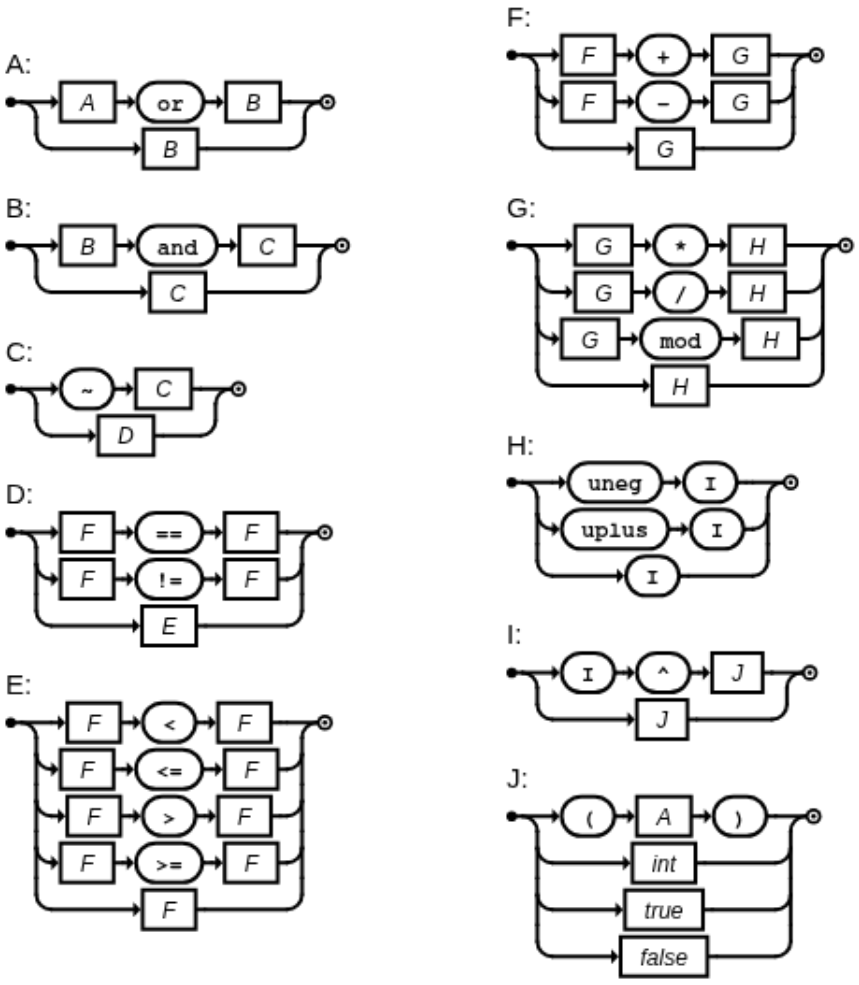
A few notes

1. Equality checks return a boolean, so allowing chains here would lead to incorrect semantics
2. Same for the other relational operators, we do not want to allow something like

   $$(a < b) < d$$

   For our purposes, we don't want to allow arithmetic or comparisons with boolean constants. If one operand is a boolean, the operator better be logical.
3. We call unary negation $\neg$, and unary plus $\oplus$

The syntax diagrams for this grammar would be



Although we do implement some semantic checks directly in the parser, it is best to allow expressions that obey syntax but disobey semantic rules to be accepted by the parser, and leave the complaining to the semantic checks after parsing is completed. However, some semantic rules are very simple to implement directly in grammar. For example, making the operands of relational operators go directly to the grammar rule that begins arithmetic expressions.

We could conceive of a grammar that does implement semantic checks in regards to operands and their requite operands. For example, in our language we require that the operands of a logical operator must be logical (true / false), and the operands of a relational or arithmetic operand must be arithmetic. In this system, it then follows that there can only be at most one relational operator per expression.

Note that the result of a logical or relational expression is a boolean, and the result of an arithmetic expression is arithmetic.

The following grammar enforces not only precedence and associativity, but also types

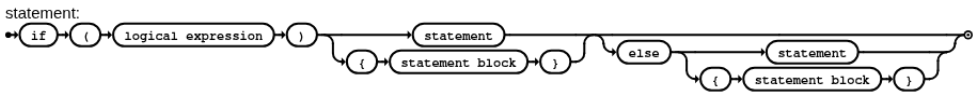
of operands.

$$
\begin{aligned}
A &\to W \mid B \\
W &\to W \text{ or } X \mid X \\
X &\to X \text{ and } Y \mid Y \\
Y &\to \sim Y \mid D' \\
B &\to D \\
D &\to F == F \mid F \neq F \mid E \\
E &\to F < F \mid F \leqslant F \mid F > F \mid F \geqslant F \mid F \\
D' &\to F == F \mid F \neq F \mid E' \\
E' &\to F < F \mid F \leqslant F \mid F > F \mid F \geqslant F \mid Z \\
Z &\to (A) \mid \text{ true } \mid \text{ false} \\
F &\to F + G \mid F - G \mid G \\
G &\to G * H \mid G/H \mid G \bmod H \mid H \\
H &\to \neg I \mid \oplus I \mid I \\
I &\to I \wedge J \mid J \\
J &\to (F) \mid \text{int}.
\end{aligned}
$$

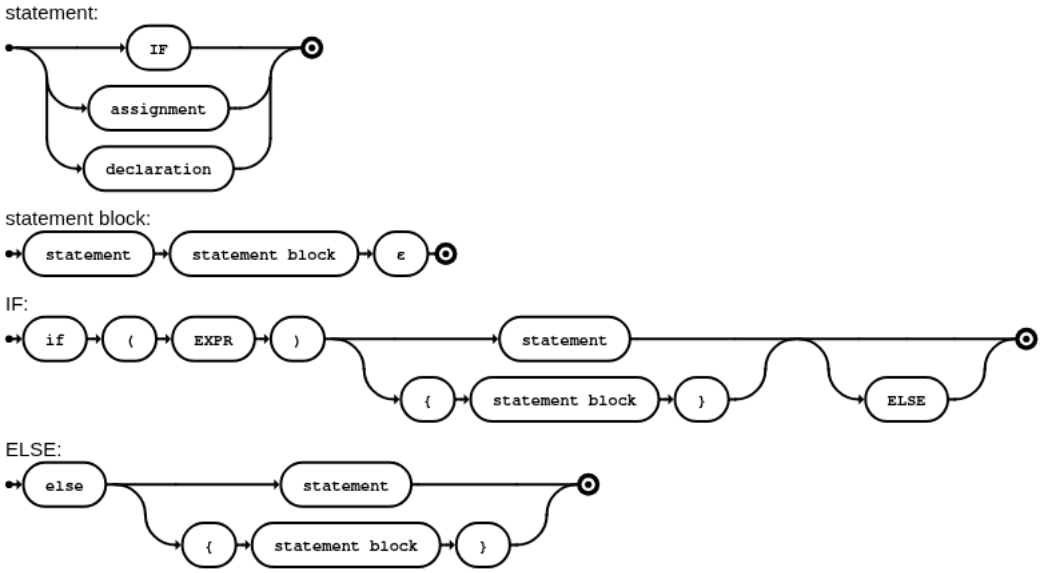
Although it's best to maintain strict separation between parsing and semantic analyzing. Let grammars / parsing shape the language, and the semantic analyzer check if statements that assume that shape make sense.

3.3 Decisions

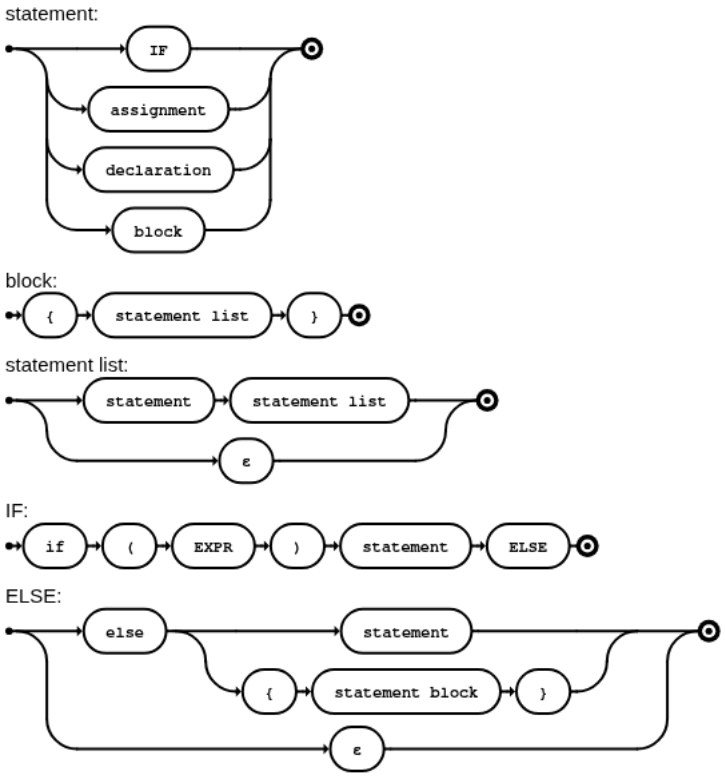
- **If statement syntax diagram**: A simple naive syntax diagram for if-else statements is



Note that a syntax diagram does not have to correspond one-to-one with individual BNF production rules. It can instead correspond to a larger syntactic category. This is essentially a simplified form of



And, conventionally statement blocks are handled in the following way



Note that we also changed how `else` is handled. Additionally, these diagrams don't account for the fact that a statement could be ε. So in the language defined above, a statement cannot be empty.

A grammar up to this point could be

$$
\begin{aligned}
\text{STMT} &\to \text{IF} \mid \text{ASSIGN} \mid \text{DECLARE} \mid \text{BLOCK} \\
\text{IF} &\to \text{if (EXPR) STMT ELSE} \\
\text{ELSE} &\to \text{else STMT} \mid \varepsilon \\
\text{BLOCK} &\to \{\text{STMTLIST}\} \\
\text{STMTLIST} &\to \text{STMT STMTLIST} \mid \varepsilon \\
\text{ASSIGN} &\to \text{ident} := \text{EXPR}; \\
\text{DECLARE} &\to \text{TYPE ident}; \\
\text{TYPE} &\to \text{int4} \\
\text{EXPR} &\to A \\
A &\to A \text{ or } B \mid B \\
B &\to B \text{ and } C \mid C \\
C &\to \sim C \mid D \\
D &\to F == F \mid F \neq F \mid E \\
E &\to F < F \mid F \leqslant F \mid F > F \mid F \geqslant F \mid F \\
F &\to F + G \mid F - G \mid G \\
G &\to G * H \mid G/H \mid G \bmod H \mid H \\
H &\to \neg I \mid \oplus I \mid I \\
I &\to I \wedge J \mid J \\
J &\to (A) \mid \text{int} \mid \text{ true} \mid \text{ false} \mid \text{ident} .
\end{aligned}
$$

Note that a statement block is itself a statement.

- **The dangling else problem**: The dangling else problem is the ambiguity that appears when your grammar allows both

```
if (EXPR) STMT
```

and

```
if (EXPR) STMT ELSE STMT
```

because then an else may have more than one possible if to attach to. For example,

```
if (a < 10)
    if (a > 2)
else
    ...
```

So, which if should the `else` be attached to? In most languages, the else will attach to the very last if. Without accounting for this problem, our grammar would be ambiguous.

One way to fix this problem is to enforce blocks after if statements. Another is to use the **MATCHED/UNMATCHED** construct.

- **Matched / unmatched construct**: This is the standard compiler-theory solution. We split statements into two categories

  1. **Matched statements**: Every `if` inside already has its `else`
  2. **Unmatched statements**: There is an `if` still waiting for an `else`

  The idea is that an `else` is only allowed to pair with the closest `if` that is still unmatched.

  A standard version is

  $$\begin{aligned} \text{STMT} &\to \text{MATCHED} \mid \text{UNMATCHED} \\ \text{MATCHED} &\to \text{ASSIGN} \mid \text{DECLARE} \mid \text{BLOCK} \mid \texttt{if (EXPR) MATCHED else MATCHED} \\ \texttt{UNMATCHED} &\to \text{if (EXPR) STMT} \mid \text{if (EXPR) MATCHED else UNMATCHED.} \end{aligned}$$

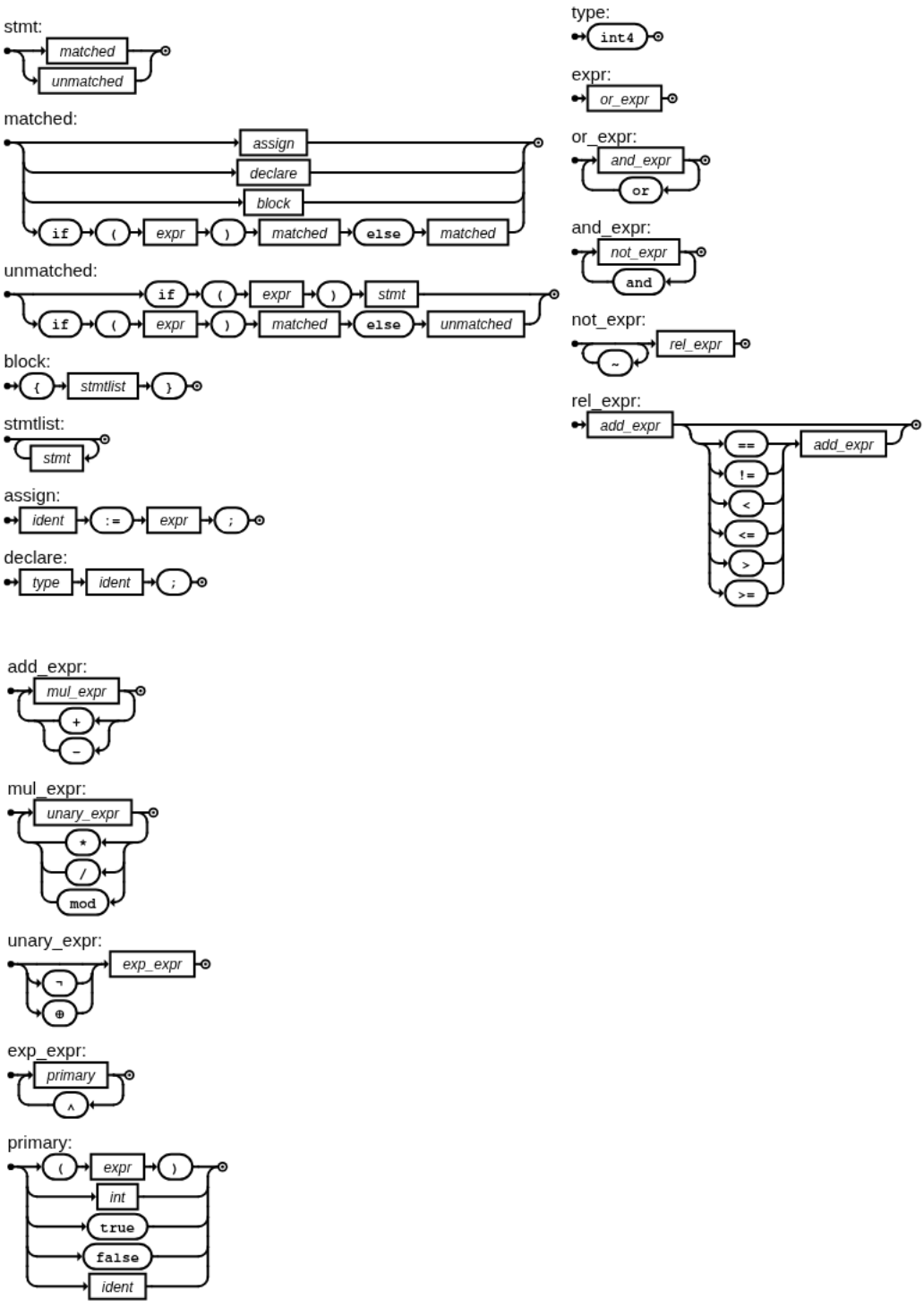
  So,

  - A statement is matched if every if in it already has a partner else.
  - A statement is unmatched if somewhere at the outer edge there is an if with no else yet.

  Now, the grammar is

  $$\begin{aligned} \text{STMT} &\to \text{MATCHED} \mid \text{UNMATCHED} \\ \text{MATCHED} &\to \text{ASSIGN} \mid \text{DECLARE} \mid \text{BLOCK} \mid \texttt{if (EXPR) MATCHED else MATCHED} \\ \texttt{UNMATCHED} &\to \text{if (EXPR) STMT} \mid \text{if (EXPR) MATCHED else UNMATCHED} \\ \text{BLOCK} &\to \{\text{STMTLIST}\} \\ \text{STMTLIST} &\to \text{STMT STMTLIST} \mid \varepsilon \\ \text{ASSIGN} &\to \text{ident} := \text{EXPR}; \\ \text{DECLARE} &\to \text{TYPE ident}; \\ \text{TYPE} &\to \text{int4} \\ \text{EXPR} &\to A \\ A &\to A \text{ or } B \mid B \\ B &\to B \text{ and } C \mid C \\ C &\to \sim C \mid D \\ D &\to F == F \mid F \neq F \mid E \\ E &\to F < F \mid F \leqslant F \mid F > F \mid F \geqslant F \mid F \\ F &\to F + G \mid F - G \mid G \\ G &\to G * H \mid G/H \mid G \bmod H \mid H \\ H &\to \neg I \mid \oplus I \mid I \\ I &\to I \wedge J \mid J \\ J &\to (A) \mid \text{int} \mid \text{true} \mid \text{false} \mid \text{ident}, \end{aligned}$$

  which produces the following syntax diagrams.



- **Else if**: We remark that the above grammar does handle `else if`, an else if (...) ... is not usually a separate primitive grammar form. It is just:

```
0  else STMT
```

Where stmt happens to be another `if`. So, something like

```
if (a) s1 else if (b) s2 else s3
```

is parsed as

```
if (a) s1 else [ if (b) s2 else s3 ]
```

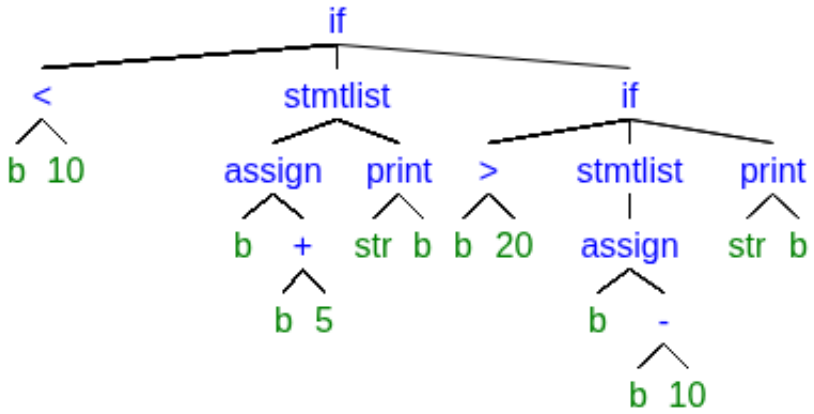
- **AST for if statements**: Consider the naive grammar that supports decision statements

$$\begin{aligned} \text{STMT} &\rightarrow \text{BLOCK} \mid \text{ASSIGN} \mid \text{DECLARE} \mid \varepsilon \\ \text{BLOCK} &\rightarrow \{\text{STMTLIST}\} \\ \text{STMTLIST} &\rightarrow \text{STMT SMTLIST} \mid \varepsilon \\ \text{IF} &\rightarrow \text{if (EXPR) STMT} \mid \text{if (EXPR) STMT else STMT} . \end{aligned}$$

Then, for

```
if (b < 10) {
    b := b +5;
    print("b: ", b);
} else if (b > 20) {
    b := b-10;
    print("else if");
} else
    print("Something else");
```

The AST would then be



3.4 Scope

- **Intro**: In compiler theory, scope is the region of a program in which a declared name is valid and can be referred to by its identifier.

  When semantic analysis uses symbol tables, scope determines where an identifier binding is visible. A binding is the association between a name and its meaning, such as:

  - variable name → type and storage information
  - function name → parameter types and return type
  - class name → type definition

  Scope lets the compiler answer questions such as:

  - Has this identifier been declared?
  - Which declaration does this use refer to?
  - Is this identifier being used outside its lifetime/visibility region?
  - Is this declaration a legal redeclaration, or is it a conflict?

  So scope checking is one of the main tasks of semantic analysis.

- **Implementing scope**: You typically implement scope by maintaining a stack of environments, where each environment is a symbol table for one lexical region.

  When the compiler enters a new block, function, loop body, or similar construct, it creates a new scope. When it leaves that construct, it destroys that scope. Name lookup searches from the innermost scope outward.

  A common design is:

  - one symbol table per scope
  - a stack of those tables
  - lookup that searches from top to bottom

3.5 Loops

- **A for loop can be translated into a while loop**: Consider

```
for (a; b; c) ...
```

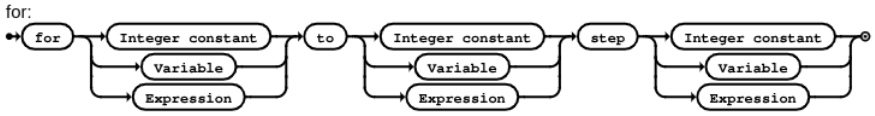
this is the same as

```
a;
while (b) {
    .
    .
    .
    c;
}
```

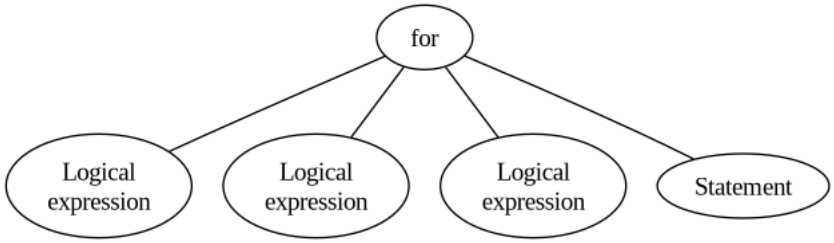
- **For loops**: Consider the syntax

```
for (val to val step val)
```

where each *val* is a integer constant, variable, or expression. The syntax diagram is then



and the AST is

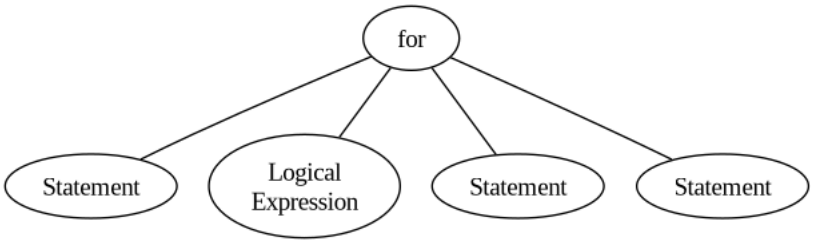


Next, consider the syntax

```
for (a; b; c) ...
```

Then, the syntax diagram is

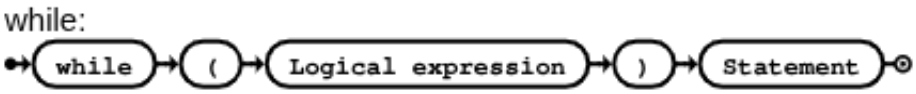
and the AST is



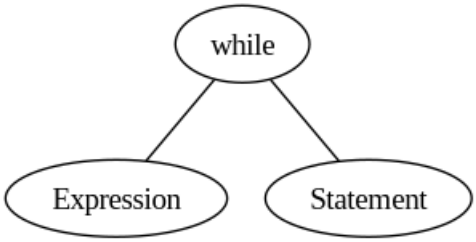
- **While loops**: Consider the syntax

```
while (Expression)
```

The syntax diagram is



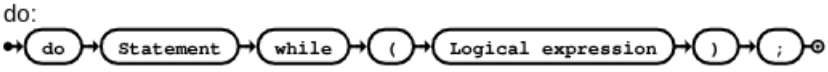
And the AST is

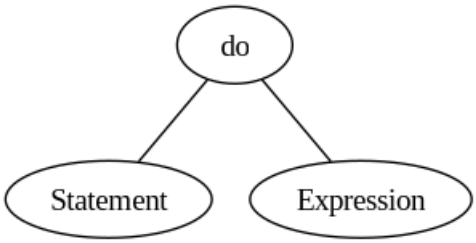


- **Do while**: Consider The syntax

```
do ... while (Expression);
```

Then, we have




do
Statement
Expression

Intel x86-64 architecture and code generation

- **General purpose registers**: General-purpose registers store integers, pointers, loop counters, and intermediate results.

  The 32-bit registers are

  - EAX – Accumulator (frequently used for arithmetic and return values)
  - EBX – Base register
  - ECX – Counter (loops, shifts)
  - EDX – Data register (multiplication/division)
  - ESI – Source index (string operations)
  - EDI – Destination index (string operations)
  - EBP – Base pointer (stack frame)
  - ESP – Stack pointer

  The 64-bit registers are

  - RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP
  - R8–R15 (additional registers introduced in 64-bit mode)

  Each 64-bit register has accessible sub-registers. For example,

  $$\text{AX} \rightarrow \text{EAX} \rightarrow \text{AX} \rightarrow \text{AH/AL}.$$

  This backward compatibility is fundamental to x86 design.

- **3-bit register specifier**: Many x86 instructions include a 3-bit register specifier, which implicitly numbers the registers:

Encoding	Register
000	EAX
001	ECX
010	EDX
011	EBX
100	ESP
101	EBP
110	ESI
111	EDI

  Note: Exists only at the binary encoding level

- **Instruction pointer (RIP)**:
  - **EIP (32-bit) / RIP (64-bit)**: Holds the address of the next instruction to execute.

- **The return instruction (C3)**: The ret (return) instruction transfers control back to the calling function by restoring the instruction pointer from the stack. It is the logical inverse of the call instruction.
- **Dynamic code execution basic example (CPP) (JIT execution)**: We will
  - Allocate memory at runtime
  - Write machine code (Intel x86/x86-64 instructions) into that memory
  - Mark the memory as executable
  - Cast the address to a function pointer
  - Call it

  This is exactly what a JIT compiler does at a minimal level. Consider

```
unsigned char prog[50000];
prog[0] = 0xC3; // ret instruction

int value;
// Cast to a function pointer, then call, store returned
 ↪  value from call in value
value = ((int(*)(void))prog)();
```

  If we run this code, we will get a seg fault. This is because our memory block is only read/write, we need a way to mark the memory as executable. We can do this by using sys/mman.h to allocate memory through the operating system, and errno to check for errors.

- **sys/mman.h**: <sys/mman.h> declares the memory-mapping API. It allows a process to request, control, and manage virtual memory regions directly from the operating system.

  The function **mmap** maps a region of virtual memory.

```
void* mmap(
    void* addr,
    size_t length,
    int prot,
    int flags,
    int fd,
    off_t offset
);
```

  fd is an open file descriptor referring to a file or device, it declares "this mapping is backed by the contents of this file." It must be opened with permissions compatible with prot. The kernel uses it as the source of bytes for the mapping. offset specifies where in the file the mapping starts, in bytes.

  Returns a pointer to the mapped region. On failure, it returns MAP_FAILED.

  munmap Unmaps a previously mapped region.

```
int munmap(void* addr, size_t length);
```

After munmap, any access to the region is invalid.

mprotect Changes access permissions on an existing mapping.

```
int mprotect(void* addr, size_t len, int prot);
```

Commonly used to transition memory:

- **RW:** Write code
- **RX:** Execute code

This is critical for enforcing $W \oplus X$ (write xor execute) security.

The protection flags (prot) are

- **PROT_READ**: Readable
- **PROT_WRITE**: Writable
- **PROT_EXEC**: Executable
- **PROT_NONE**: No access

These specify what the CPU is allowed to do with the memory

The mapping flags (flags) are

- **MAP_PRIVATE**: Copy-on-write
- **MAP_SHARED**: Shared between processes
- **MAP_ANONYMOUS**: Not backed by a file
- **MAP_FIXED**: Place mapping at a fixed address (dangerous)

- **errno**: errno is a thread-local integer that reports why a system call failed. It is declared in

```
<errno.h>
```

System calls signal failure via return values, they set errno to a symbolic error code. The value persists until overwritten

```
void* p = mmap(...);
if (p == MAP_FAILED) {
    perror("mmap");
}
```

Only meaningful immediately after failure, not reset automatically. The common errno values with mmap are

- **ENOMEM**: Insufficient memory or address space
- **EACCES**: Permission denied (e.g., PROT_EXEC disallowed)
- **EINVAL**: Invalid flags, alignment, or parameters
- **EBADF**: Invalid file descriptor
- **ENODEV**: Unsupported mapping type

To convert errno to text, use perror or strerror

```
perror(const char*)
strerror(errno)
```

perror prints a textual description of the error code currently stored in the system variable errno to stderr. It's parameter is a pointer to a null-terminated string with explanatory message

- **DCE with mmap and errno**:

```
const int prog_size = 50000;
unsigned char* prog;

   prog = mmap(0, prog_size, PROT_EXEC  | PROT_READ  |
↪  PROT_WRITE, MAP_PRIVATE  | MAP_ANONYMOUS, -1, 0);

if (errno) {
    perror("mmap");
    return;
}
```

Having fd=-1 means "This mapping is not backed by any file." This is only valid when used together with MAP_ANONYMOUS. Without it, fd=-1 is an error.

Having addr=0 means that the memory placement is decided by the operating system.

Now, prog points to the start of our block of memory that is readable, writable, and executable. Remember, to free the memory we must call `munmap`

```
munmap(prog, prog_size);
```

Now, with

```
value = ((int(*)(void))prog)();
```

We can output the value of value, which will give us the current value in the accumulator RAX. The return value of a function is defined to be placed in the accumulator register

- prog points to executable memory
- (int(*)(void))prog reinterprets that address as a function
- () performs a call instruction
- The CPU jumps to prog
- Your machine code executes
- A ret instruction returns control to the caller
- The caller reads the return value from the accumulator
- That value is assigned to value

Remember that we have

```
prog[0] = 0xC3
```

I.e the ret instruction, which is the only instruction currently in our program.

```
cout << "Memory at: " << (int*) prog << '\n';
```

Note that if we left our program pointer as char*, C++ uses the overload for outputing C-style strings. Since we just want the address of the first byte of our program, we must cast it to a different pointer, in this case a pointer to an integer. A better way to go about doing this is

```
cout << "Memory at: " << static_cast<void*>(prog) << '\n';
```

- **The ModR/M byte**: The ModR/M byte is a mandatory operand-encoding byte used by many x86 instructions. It specifies which registers are used and whether an operand refers to a register or to memory, including the basic addressing mode.

  The ModR/M byte is one byte (8 bits), divided into three fields

  - **r /m**: Bits 0-2
  - **reg**: Bits 3-5
  - **mod**: Bits 6-7

  mod determines how the r/m field is interpreted:

  - **00**: Memory operand, no displacement (except special cases)
  - **01**: Memory operand + 8-bit displacement
  - **10**: Memory operand + 32-bit displacement (16-bit in 16-bit mode)
  - **11**: Register operand (no memory access)

  Note that if we had a displacement, then that displacement would go after the ModR/M byte in the encoding.

The reg field usually selects a register operand.

  - **000**: EAX
  - **001**: ECX
  - **010**: EDX
  - **011**: EBX
  - **100**: ESP
  - **101**: EBP
  - **110**: ESI
  - **111**: EDI

In /r encodings, reg is a real operand. In /0–/7 encodings, reg is an opcode extension, not a register

The r/m field selects:

  - A register if mod = 11
  - A memory addressing mode if mod $\neq$ 11

In the second case, r/m selects a base addressing form. In 32/64-bit mode:

  - 000 [EAX]
  - 001 [ECX]
  - 010 [EDX]
  - 011 [EBX]
  - 100 SIB byte follows
  - 101 [disp32] if mod=00, otherwise [EBP]+disp
  - 110 [ESI]
  - 111 [EDI]

If r/m = 100 and mod $\neq$ 11, a SIB (Scale–Index–Base) byte follows. This allows

$$[\text{base} + \text{index} \cdot \text{scale} + \text{displacement}].$$

- **Word sizes in Intel architecture**:
  - **Byte**: 8 bits
  - **Word**: 16 bits
  - **Doubleword (dword)**: 32 bits
  - **Quadword (qword)**: 64 bits
  - **Double quadword (dqword / xmmword)**: 128 bits
  - **YMM word**: 256 bits
  - **ZMM word**: 512 bits
- **MOV instructions: Putting a value into the accumulator**: For this we use the instruction

```
MOV     dst,src
```

Many forms of the MOV instruction can be found in the Intel manual. There are different forms based on whether dst, src is memory or a register, and how many bits these fields are. For example, we could use

```
MOV    r/m8,r8
```

where r/m8,r8 means that the destination operand can be either an 8-bit memory location, or an 8-bit register, and the source must be an 8-bit register. Conversely,

```
MOV    r8,r/m8
```

is the opposite. In the first form, the opcode is 88 /r, where /r denotes that the encoding requires a modR/M byte.

Note that there are forms for 8, 16, 32, and 64 bit. There are also immediate byte forms, for example

```
MOV    rk,immk
```

where $k \in \{8, 16, 32, 64\}$. Consider the form

```
MOV r32,imm32
```

This form has opcode $B8+$ rd id, where the d stands for *double word* 32-bit. If our instruction is

```
MOV    EAX,4
```

where 4 is the immediate byte, then the opcode is

$$B804000000.$$

Note that EAX is 000, so B8 remains the same. Further note that our immediate byte is 32-bits, since we used the double word version, and those bytes are in little endian, so the bytes are reversed. In BE it would be

$$00000004.$$

Another note is that there are no memory to memory moves, we must go memory to register, then register to memory.

Let's add this MOV instruction to our code, we add to the program buffer

```
size_t p_offset = 0;

prog[p_offset++] = 0xb8; // MOV EAX,4
prog[p_offset++] = 0x04;
prog[p_offset++] = 0x00;
prog[p_offset++] = 0x00;
prog[p_offset++] = 0x00;
prog[p_offset++] = 0xc3; // RET to caller
```

Now, the value returned from the call to prog gives the value 4, as 4 is the value in the accumulator.

- **A note**: The 64-bit architecture is called IA-32e, since it is an extension of the 32-bit architecture, not a 64-bit rework.
- **Two's complement**: IA uses twos complement, so to put -7 into the accumulator, we use

  $$f9ffffff.$$

  as the immediate byte. Notice the LE.
- **Finite width arithmetic**: An n-digit base-b register can store exactly

  $$\{0, 1, \ldots, b^n - 1\},$$

  nothing else. Any computation that exceeds this range wraps. The hardware drops the carry out of the top digit. This behavior is precisely arithmetic modulo b^n.
- **Twos complement**: This system comes from the question... On an n-digit base b-machine, what value should represent $-x$ so that subtraction can be done by addition.

  So, with finite-width arithmetic, we want a value y such that

  $$x + y \equiv 0 \pmod{b^n}.$$

  So, y is the additive inverse of x modulo b^n. Notice that since $x + y \equiv 0 \pmod{b^n}$,

  $$x + y = kb^n$$

  for some integer k. However, since

  $$0 \leqslant x, y \leqslant b^n - 1$$

  implies that

  $$0 \leqslant x + y \leqslant 2(b^n - 1) = 2b^n - 2,$$

  the only multiples of b^n that $x + y$ can equal are 0 and b^n, so we can safely say that $k = 1$. Thus, our equation becomes

  $$x + y = b^n.$$

  From this, we can solve for y

  $$y = b^n - x.$$

  Therefore,

  $$x + (b^n - x) = b^n,$$

  since $x + y = b^n$, and $y = b^n - x$. Thus,

  $$x + (b^n - x) \equiv 0 \pmod{b^n}.$$

- **Why twos complement works**: Consider an n digit system (base b), and a number x in that system. If we subtract by b^n, we get

$$b^n - x.$$

Now, we can add zero to this subtraction

$$b^n - 1 - x + 1 = (b-1)(b^{n-1} + b^{n-2} + \cdots + b^0) - x + 1.$$

Notice that $b^n - 1$ becomes $(b-1)(b^{n-1} + b^{n-2} + \cdots + b^0)$. Further notice that $b^n - 1$ is the number with all digits equal to $b - 1$. In base 2, this would be all bits set to 1.

This algebraic trick reveals that

$$b^n - 1 = (b-1)(b^{n-1} + \cdots + b^0).$$

From this, subtracting x gives the digitwise complement of x. In base two, this would mean flipping all the bits. Thus,

$$b^n - 1 - x + 1$$

is the digitwise complement plus one, where the plus one is a carry. This is the step that turns the digitwise complement into the true additive inverse in base b. We now understand that

$$\bar{x} = (b^n - 1) - x.$$

This is the digitwise (radix-$b-1$) complement. So, we know the complement and the additive inverse. Notice that all we need to do to turn the complement into the additive inverse is add one to the complement.

- **A cleaned up twos complement argument**: Consider a base-b, n-digit system, note that our context is finite width arithmetic, since this is how registers work. Thus, all arithmetic is taken modulo b^n, since

$$0 \leqslant x \leqslant b^n - 1$$

for all x in our system. Now consider

$$b^n - x$$

Now, add zero to this expression

$$b^n - 1 - x + 1.$$

Consider the quantity $b^n - 1$, we can factor out a $b - 1$ to get

$$(b^n - 1) = (b-1)(b^{n-1} + b^{n-2} + \cdots + b^1 + b^0).$$

Observe that this quantity is a number in our system with all digits equal to $b-1$. For example, if $b = 2$, then this is precisely the expression that gives us an n-bit binary number with all bits set to one.

So, subtracting x from this quantity gives the radix $b - 1$ (digitwise) complement. In base two this would mean flipping all bits in x. So,

$$b^n - 1 - x + 1$$

is the digitwise complement plus one. Our goal is to derive the additive inverse of x using this digitwise complement. The additive inverse of x in unbounded width arithmetic would be the number y such that

$$x + y = 0.$$

However, we are dealing with finite width arithmetic, so all expressions equal to zero must be congruent to zero when taken modulo b^n. So, we must instead consider

$$x + y \equiv 0 \pmod{b^n}.$$

Using modular arthritic, this means that

$$x + y = kb^n$$

for some $k \in \mathbb{Z}$. But, notice that since

$$0 \leqslant x, y \leqslant b^n - 1,$$

it must be that

$$0 \leqslant x + y \leqslant 2(b^n - 1) = 2b^n - 2.$$

Since the maximum in our system is b^n, the only possible multiple of b^n is b^n itself, so $k = 1$ (actually $k \in \{0, 1\}$, but $k = 0$ only if $x = y = 0$, otherwise $k = 1$). Thus,

$$x + y = b^n.$$

Now we see that the additive inverse y is

$$y = b^n - x.$$

Notice that the digitwise complement $(b^n - 1 - x)$ and the additive inverse $(b^n - x)$ look very similar. All we need to do to get from the digitwise complement to the additive inverse is add one to the complement, which is precisely what twos complement does.

The story of the sign bit (in base two) arises once you define the signed interpretation map

$$\phi(u) = \begin{cases} u, & 0 \leqslant u < 2^n - 1 \\ u - 2^n, & 2^{n-1} \leqslant u < 2^n \end{cases}.$$

Notice that

$$\phi(2^n - u) = (2^n - u) - 2^n = -u = -\phi(u).$$

Any value greater than or equal to 2^{n-1}, which corresponds to MSB=1 is negative, hence the sign bit.

In addition, consider performing twos complement twice on any number x,

$$2^n - (2^n - x) = x$$

so we get back x. This is why we don't have to reverse the process of twos complement to undo it.

- **ADD instructions: Adding the value from a register into the accumulator**: Suppose we want to add +3 into ECX, then add that value to the accumulator. We can use the ADD instruction version

```
ADD    r32, r/m32
```

with opcode 03 /r and the same immediate byte move instruction

```
MOV    ECX, 3
```

This has encoding

$$B903000000.$$

The ADD instruction is

```
ADD    EAX,ECX
```

The ModR/M byte for the ADD instruction is $C1$, since mod=11, reg=000 (ECX), and r/m = 001 (EAX). Thus, the encoding is

$$03C1.$$

- **Note on instruction length**: An x86 instruction can be anywhere from 1 to 15 bytes long. There is no fixed instruction width. The length depends on which optional components are present.
- **/0 - /7**: In Intel x86 architecture, the symbols /0 through /7 refer to opcode extensions encoded in the ModR/M byte of an instruction. They are not registers themselves; rather, they select which operation an instruction performs when multiple operations share the same primary opcode.

  Many x86 instructions reuse a single opcode and rely on the ModR/M byte to disambiguate the operation. When documentation writes /0, /1, …, /7, it means "The reg/opcode field of the ModR/M byte must contain this value." That 3-bit field can encode values from 0 to 7, hence /0–/7.

  Therefore, the 3-bit reg field has exactly one meaning per instruction:

  - Either it selects a register operand
  - Or it selects an opcode extension (/0–/7)
- **Substraction instructions**: Suppose we want to subtract ECX from EAX,

```
SUB    EAX, ECX
```

Suppose ECX has value 3, and EAX has value -7. The opcode for this SUB instruction

```
SUB r/m32, r32
```

Is 29 /r. The encoding is therefore 29C8, since the ModR/M byte is C8 (11 001 000).

- **Multiplication instructions**: We have two primary instructions, MUL (unsigned multiplication), and IMUL (signed multiplication). Regarding IMUL, there are four one operand instructions, one for each register/memory size. Consider

```
IMUL    r/m32
```

It should be noted that the operand is multiplied against the value in the accumulator, and the DX/AX registers are combined to make one 64-bit register split into two 32-bit sections, 32-bits per half. The DX register is the high order 32-bits, and the AX register is the low order 32-bits. Since we will use the 32-bit version, EDX and EAX are our register pairs.

The opcode for this instruction is f7 /5, so we use 101 in the 3-bit reg field in the ModR/M byte.

If our instruction was

```
IMUL    ECX
```

,this byte would be 11 101 001, which is E9 in hex. Thus, the encoding is $F7E9$.

- **Division and converting instructions**: We have DIV for unsigned division, and IDIV for signed division. This operation divides the value in the accumulator by the value in the supplied operand, and again the result is split into the DX / AX pair. The quotient of the operation will be housed in AX, and the remainder in DX.

Before we can proceed with the division, we must extend the sign bit of the accumulator into the DX register. For this, we need CWD, CDQ, CQO instructions. CWD converts a word to a double word, and CDQ converts a double word to a quadword. In particular, the version of these instructions that extend the sign bit of AX into DX are given by opcode 99, and they take no operands. Thus,

```
CDQ
```

Extends the sign bit of EAX (32-bit doubleword) into EDX, thus converting to a quadword (64-bit). This operation must be done before division.

Note that the encoding for CDQ is 99, and the encoding for IDIV is f7 /7, where the instruction is

```
IDIV   r/m32
```

- **XOR instruction, Clearing registers**: The XOR instruction is

```
XOR   r /m32 r32
```

and variants. The opcode for this particular instruction is $31/r$. To clear a register, we therefore use the same register for both operands

- **R8-R15, the REX prefix**: The REX prefix is a one-byte instruction prefix introduced with x86-64 to extend the original x86 encoding so that it can address 64-bit operands and additional registers.

  The REX prefix is one byte, with the following fixed structure:

  ```
  0100WRXB
  ```

  The high nibble 0100 identifies it as a REX prefix. The low nibble contains four 1-bit flags.

Bit	Name	Purpose
W	REX.W	64-bit operand size
R	REX.R	Extends ModR/M reg field
X	REX.X	Extends SIB index field
B	REX.B	Extends the r/m field in ModR/M, or the base field in SIB, or an opcode-embedded register field like B8+rd

  REX.W = 1 forces a 64-bit operand size. Requires REX.W Without it, the instruction would operate on 32-bit registers.

  REX.R, REX.X, and REX.B extend 3-bit register fields into 4-bit fields.

  - **REX.R**: Extends the reg field of the modR/m byte. Place the high order bit of the non r /m register mask into this bit. For example, for R8 = 1000, set R=1.
  - **REX.X**: Extends the index field in the SIB byte, only meaningful if a SIB byte is present, otherwise leave zero
  - **REX.B**: Extends the r /m field of the modr /m byte. Place the high order bit of the r / m register mask into this bit.

  Note that REX.B also deals with a opcode-embedded register field like B8+rd. So, for an instruction like

  ```
  REX.W + b8+rd io mov r64,imm64
  ```

REX.W = 1, REX.R = 0, REX.X = 0, and REX.B is the high order bit of r64.

- **Recall: Fast exponentiation**: Uses the identity that

$$a^b = \begin{cases} (a^2)^{\frac{b}{2}} & b = 2k \\ a \cdot a^{b-1} & b = 2\ell + 1 \end{cases}.$$

```
e = 1
// b!=0 and b>=0 (nonnegative b) implies b>0
while (b>0) {
    if (b & 0x01) {
        e = e * a
    }
    a = a*a
    b>>=1 // Divide by two
}
```

- k **child trees to binary trees**: When creating abstract syntax trees, it is likely the case that a binary tree is too restrictive, we may need to have trees that can have k children, for k not restricted to 2. Thus, it is helpful to know the way in which we can convert such a tree into a binary tree.

  The idea is that for each parent node, we take its first (leftmost) child and make that its child in the binary tree. Then, for any other children (siblings of the first child), we attach those in a link to the first child. So, if A has children B, C, D, and B has children E, F, then the binary tree would have connections

$$\begin{aligned} A &\to B, \\ B &\to C, E \\ C &\to D, \\ E &\to F. \end{aligned}$$

- **The POP and PUSH instructions**: The PUSH instruction pushes a word, doubleword, or quadword onto the stack. Decrements the stack pointer and then stores the source operand on the top of the stack.

```
PUSH    r32
```

  The POP instruction loads the value from the top of the stack to the location specified with the destination operand

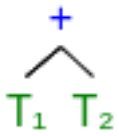
```
POP     r32
```

- **The ECHG (exchange) instruction**: Exchanges register / memory with register
- **Machine execution model (MEM)**: There is a problem that arises when trying to generate machine code. We have many registers, but when generating code, which registers do we use? And if we have to go to memory if those registers are in use, which addresses do we go to? And how does this access of memory to store values work?

Since this is outside the scope of our current progress in compiler theory, we will start with the use of a stack based model.

In this model, we will have the top of the stack always be the current value in the accumulator.

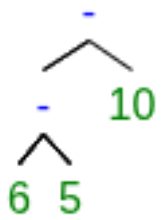
First, consider a generic tree



where T_1 and T_2 are subtrees. When T_1 completes its duties, its computed value will be in the accumulator. So, we push this value onto the stack. Then, we can generate code for T_2 and its value will now be in the accumulator. From here, we need to perform the addition with these two values. So, we pop the top of the stack into ECX, then write the instruction

```
o   ADD     EAX,ECX
```

Consider the expression $6 - 5 - 10$, which generates the AST



The code will look something like

```
// Computes 6-5 and places result on the top of the stack
MOV   EAX,6
PUSH  EAX
MOV   EAX,5
POP   ECX
XCHG  EAX, ECX
SUB   EAX, ECX
PUSH  EAX

// Computes 6-5-10, 6-5 is on the top of the stack at this point
MOV   EAX,10
POP   ECX
XCHG  EAX,ECX
SUB   EAX,ECX
```

Important note: For JIT code that ends with RET, the total number of pushes must equal the total number of pops. In 64-bit mode (IA-32e), RET always pops its return address from the current stack pointer (RSP). If your JIT code has done a net extra push (i.e., stack not balanced), then RET will pop your arithmetic value (e.g., 10) and try to jump to address 0x000000000000000A, instant crash.

- **FLAGS (EFLAGS and RFLAGS)**: EFLAGS and RFLAGS are the status/control flag registers of x86.

  They hold individual bit flags that record arithmetic results, processor state, and certain control settings.

  These names come from register width across x86 generations.

  - FLAGS = original 16-bit register on 8086
  - EFLAGS = 32-bit version in 32-bit x86
  - RFLAGS = 64-bit version in x86-64 / IA-32e

  Conceptually, they are the same logical register extended over time. In 64-bit mode, almost all meaningful flag bits are still in the low portion; the upper bits of RFLAGS are mostly reserved.

  1. **Status flags**: These reflect the result of arithmetic/logical operations.
     - CF - Carry Flag
     - PF - Parity Flag
     - AF - Auxiliary Carry Flag
     - ZF - Zero Flag
     - SF - Sign Flag
     - OF - Overflow Flag
  2. **Control flags**: These reflect the result of arithmetic/logical operations.
     - DF - Direction Flag
     - IF - Interrupt Enable Flag
     - TF - Trap Flag
  3. **System / Privilege flags**: These are used by the operating system, virtualization, tasking, privilege transitions, and related machinery

  - IOPL
  - NT
  - RF
  - VM
  - AC
  - VIF
  - VIP
  - ID

  Only some flags have dedicated instructions to directly set or clear them. Many do not, and are instead changed only as a side effect of other instructions, or only by restoring a saved flags value.

  - CLC - clear carry flag (CF := 0)
  - STC - set carry flag (CF := 1)
  - CMC - complement carry flag (CF := ∼CF)
  - CLD - clear direction flag (DF := 0)
  - STD - set direction flag (DF := 1)
  - CLI - clear interrupt flag (IF := 0)
  - STI - set interrupt flag (IF := 1)

  We can push / pop the FLAGS register with `pushfq, popfq`. Even though POPFQ exists, not every bit in RFLAGS can necessarily be changed arbitrarily:

  - some bits are reserved
  - some bits are fixed
  - some bits are privileged
  - some bits may ignore writes in user mode

  So pushfq / popfq gives you a whole-register interface, but not unrestricted control over every bit.

- **The TEST instruction**: The TEST instruction in IA-32e (the 64-bit extension of the x86 architecture) performs a bitwise logical AND between two operands without storing the result. Its sole purpose is to update the condition flags in RFLAGS. Given

  ```
  TEST  op1, op2
  ```

  The processor computes

  ```
  temp := op1 AND op2
  ```

  The value of temp is not written back to any operand. Instead, only flags are updated. After execution:

  - **ZF (Zero Flag)**: Set if temp == 0, otherwise cleared.
  - **SF (Sign Flag)**: Set according to the most significant bit of temp.
  - **PF (Parity Flag)**: Set according to parity of the low byte of temp.

- **CF (Carry Flag)**: Cleared to 0
- **OF (Overflow Flag)**: Cleared to 0
- **AF (Auxiliary Flag)**: Undefined

This is identical to AND except the result is discarded.

- **Common use cases of TEST**:

1. **Checking if a register is zero**:

```
TEST   R,R

// Jump on ZF zero
jz
```

2. **Testing specific bits**:

```
test rax, 1
```

This tests if the LSB is set (check ZF).

- **A quick note about registers r8-r15**: In IA-32e mode, the registers R8 through R15 are full 64-bit general-purpose registers. Like

$$\text{RAX} \rightarrow \text{EAX} \rightarrow \text{AX} \rightarrow \text{AL},$$

the extended registers also have smaller views

Size	Name Example
64-bit	R8
32-bit	R8D
16-bit	R8W
8-bit	R8B

Using a smaller view automatically clears the unused upper regions, the same is true for the named registers.

Any instruction referencing R8–R15 uses a REX prefix (IA-32e encoding extension). Without it, the encoding refers to legacy registers.

Note: We can use these registers with r32 instructions, although will still need a REX prefix. In this case, the 32-bit is the operand size, and we should set REX.W = 0. Although the explicit instruction would use the doubleword subregister, we can still use Rk informally. For example,

```
INST   r8,r9
```

could use INSTs r32 variant, with a REX prefix and REX.W = 0, which is effectively the same as

```
INST   r8d, r9d
```

assuming we want our operands to be 32-bit.

- **Compare instruction**: The compare instruction perform a subtraction

$$\mathrm{op}_1 - \mathrm{op}_2$$

but does not store the result anywhere. It only updates the status flags in RFLAGS, and those flags are then used by conditional jumps like JE, JL, JG, JB, and so on.

- **SETcc**: SETcc is the x86 family of set-on-condition instructions. It evaluates a condition based on the current flags in EFLAGS / RFLAGS, and writes:
  - 1 if the condition is true
  - 0 if the condition is false

  into a **single byte operand**

```
cmp eax, ebx
sete al
```

sets AL = 1 if they are equal, zero otherwise.

- **Fast exponentiation**: The FE algorithm leverages the idea that for $a \in \mathbb{Z}$, $b \in \mathbb{Z}_{\geqslant 0}$,

$$a^b = \begin{cases} (a^2)^{\frac{b}{2}} & b \in 2k,\ k \in \mathbb{Z}, \\ aa^{b-1} & b \in 2\ell + 1,\ \ell \in \mathbb{Z} \end{cases}.$$

Which therefore gives us exponentiation in logarithmic time.

```cpp
typedef long long ll;
ll _fe(ll a, ll b) {

    if (b <= 0) {
        cerr << "Negative b rejected'\n";
        _exit(1);
    }

    int e{1};
    while (b != 0) {
        // Check if the LSB is set, then b is odd, so factor
        ↪ out a
        if (b & 1) {
            e *= a;
        }

        // Square a
        a *= a;

        // Divide b by two
        b>>=1; // floors b/2, no need to subtract one from b
        ↪ to make it even
    }
    return e;
}
```

- **Code generation for the fast exponentiation algorithm**: We aim to compute the quantity $e = a^b$. For this, we will place

$$e \to r8, \quad a \to r9, \quad b \to r10.$$

Assume the stack has b on the bottom, and a at the top. The code is then

```
// Clear out r8
4531C0   XOR   r8,r8

// Place |a| in r9
418FC1   POP   r9

// Place |b| in r10
418FC2   POP   r10

// Check if b<0, reject b by jumping to end
4585D2   TEST  r10,r10
7C1E     JL    30

// Increment r8 by one (set e=1)
41FFC0   INC   r8

// Test b==0, if true jump to end
4585D2   TEST  r10, r10
7416     JE    22

// Test if LSB set on b, jump over first IMUL if ZF equal to
↪  zero (b & 1 == 0) (b even)
41F7C2   TEST  r10,1
7404     JE    4

// Multiply e by a (factor out a) (e *= a)
450FAFC1 IMUL  r8,r9

// Square a
450FAFC9 IMUL  r9,r9

// Shift arithmetic right one (floor(b/2))
41D1FA   SAR   r10,1

// Jump back to top of loop
EBE5     JMP    -27

// Move result into accumulator
418BC0   MOV   EAX,r8
```

Note: Generated machine code on the left for each instruction. To use this code, we first need to

```
MOV   ECX,b
MOV   EAX,a

PUSH  ECX
PUSH  EAX
```

- **Instruction explanations**

– **Jumps**: The JL instruction

$$75\ \text{cb},$$

where cb stands for *code byte*, and is the number of bytes you want to jump. Byte zero is the start of the instruction following the jump instruction. So, start at the next instruction, and use the encodings to count the number of bytes until jump point.

In the last jump, we need to jump backwards, so the byte count is -27 = E5.

– **IMULS**: The version of IMUL above is

```
0  0F AF /r   IMUL r32, r/m32
```

which is defined as

$$\text{dword register} := \text{dword register} * r/m32.$$

So, instead of using the DX / AX pair, we just multiply the contents of two 32-bit registers and hope the result fits in a 32-bit register.

- **The RIP (instruction pointer)**: In IA-32e mode (Intel 64-bit architecture), the program counter is a 64-bit register known as the RIP (Instruction Pointer). It holds the memory address of the next instruction to be executed. The RIP is implicitly updated by the processor during execution and cannot be directly accessed by software, but is modified via control-transfer instructions.
- **More conditional jumps (defined in section JCC of the IA-32e manual)**:

Instruction	Meaning	Opcode
JE / JZ	Jump if equal / zero (ZF=1)	74 cb
JNE / JNZ	Jump if not equal / not zero (ZF=0)	75 cb
JL / JNGE	Less than (signed)	7C cb
JLE / JNG	$\leqslant$ (signed)	7E cb
JG / JNLE	$>$ (signed)	7F cb
JGE / JNL	$\geqslant$ (signed)	7D cb
JB / JC / JNAE	Below (unsigned $<$)	72 cb
JBE / JNA	$\leqslant$ (unsigned)	76 cb
JA / JNBE	$>$ (unsigned)	77 cb
JAE / JNB / JNC	$\geqslant$ (unsigned)	73 cb
JS	Sign flag set	78 cb
JNS	Sign flag clear	79 cb
JO	Overflow	70 cb
JNO	No overflow	71 cb
JP / JPE	Parity even	7A cb
JNP / JPO	Parity odd	7B cb
JCXZ / JECXZ / JRCXZ	Jump if CX/ECX/RCX = 0	E3 cb

Note: A short jump is a 8-bit relative branch instruction. So, we can jump 127 bytes forward and 128 bytes backwards. The jump is relative to the RIP (instruction pointer)

- **Unconditional jump (JMP)**: The short unconditional jump instruction is

```
EB cb   JMP rel8
```

- **Shifts (with immediate bytes)**:
  - **Shift arithmetic left (SAL)**: Multiply r/m32 by 2, imm8 times.

```
C1 /4 ib SAL r/m32, imm8
```

    There is also an instruction if you need the immediate byte to be one.

```
D1 /4 SAL r/m32, 1
```

  - **Shift arithmetic right (SAR)**: Signed divide r/m32 by 2, imm8 times.

```
C1 /7 ib SAR r/m32, imm8
```

and

```
D1 /7 SAR r/m32, 1
```

- **Shift logical left (SHL)**: Multiply r/m32 by 2, imm8 times.

```
C1 /4 ib SHL r/m32, imm8
```

and

```
D1 /4 SHL r/m32,1
```

- **Shift logical right (SHR)**: Unsigned divide r/m32 by 2, imm8 times.

```
C1 /5 ib SHR r/m32, imm8
```

and

```
D1 /5 SHR r/m32, 1
```

- **Increment (INC)**: Increment r/m doubleword by 1.

```
FF /0 INC r/m32
```

- **The CALL instruction**: We will use the versions

```
FF /2 CALL r/m16
FF /2 CALL r/m32
FF /2 CALL r/m64
```

which require us to load an address into a register, and then use that register as the operand. There are also versions

```
E8 cw CALL rel16
E8 cd CALL rel32
```

which allow us to specify the number bytes relative to the RIP.

- **Loading values into our program buffer**: Consider the case in which we have a 64-bit value in cpp, and we wish to place this value into our program buffer. Recall that each entry in the buffer is 8-bit. So, our value will take up the next 8 available bytes in the buffer.

```
0  void load_imm64(unsigned char* prog, int& p_offset, long
   ↪  long value) {
1      for (int i=0; i<8; ++i) {
2          // Grab the right most 8-bits, place in buffer
3          prog[p_offset++] = value & 0xff;
4
5          // Right shift 8 bits to place the next 8-bits in the
           ↪  8 low order bit position for next iteration.
6          value>>=8;
7      }
8  }
```

Then, suppose we want to place

```
0  // Some random int64
1  long long useful_int64 = 0x80C24FA11AF32C08;
```

Then,

```
0  load_imm64(prog, p_offset, useful_int64);
```

Places this value into our program buffer. This exercise is useful if, for example, we require an address placed into our program buffer to be used as an operand.

- **Simple I/O**: Instead of dealing with the complexities of low-level I/O, we will instead create high-level C/CPP functions, and use the IA-32e instruction set to call those functions, using registers for passed arguments and the accumulator for return values.

Consider the function

```
0  void print_int(int p) {
1      std::cout << p;
2  }
```

Now, suppose we have a program that places some value into EAX, some other value into ECX, and uses a subtract instruction to find the difference. This difference would then be in the accumulator, which we can easily retrieve, but what if we want to instead output the result to stdout?. So, at this moment we have

```
.
.
.

prog[p_offset++] = 0xb8;  // MOV EAX, -7
prog[p_offset++] = 0xf9;
prog[p_offset++] = 0xff;
prog[p_offset++] = 0xff;
prog[p_offset++] = 0xff;
prog[p_offset++] = 0xb9;  // MOV  ECX, 3
prog[p_offset++] = 0x03;
prog[p_offset++] = 0x00;
prog[p_offset++] = 0x00;
prog[p_offset++] = 0x00;
prog[p_offset++] = 0x29;  // SUB  EAX, ECX
prog[p_offset++] = 0xc8;
```

From here, we simply need to move the address of `print_int` into some register, move the argument for `print_int` into some other register, and use the call instruction with the operand being the register that contains the address of `print_int`.

```
MOV  RSI, print_int
MOV  EDI, EAX
CALL RSI
```

Note: The System V ABI dictates that the first six integer or pointer arguments are passed in

- RDI, RSI, RDX, RCX, R8, and R9

in that order. This is why our argument was placed into EDI. The functions address being placed into ESI was not determined by any conventions.

For the first instruction, we use the 64-bit MOV, since our functions address is 64-bit:

```
REX.W + B8+rd io MOV r64, imm64
```

where io means immediate octaword. RSI maps to six, so REX.W + B8+rd is

$$01001000\ B8 + 6 = 48\ BE.$$

Note: REX.W + B8 means prepend REX to B8 with REX.W on.

So, immediately after `prog[p_offset++] = 0xc8` in our code above, we put

```
prog[p_offset++] = 48; // MOV RSI, print_int
prog[p_offset++] = B8;
load_imm64(prog, p_offset, print_int); // Place the address
    ↪  of print_int in the next 8 bytes of the buffer, since the
    ↪  CPU will look at these bytes when looking for the operand
    ↪  of our MOV
```

Then, the second and third instructions use

```
89 /r MOV r/m32, r32
FF /2 CALL r/m64
```

So, the encodings are

$$89C7 \quad FFD6$$

respectively. Thus, we append to the program

```
prog[p_offset++] = 89; // MOV EDI, EAX
prog[p_offset++] = C7;
prog[p_offset++] = FF; // CALL RSI
prog[p_offset++] = D6;

prog[p_offset++] = C3; // RET
```

- **Return values in call instructions**: Suppose we have

A fn(int x) .

and we want to write a call instruction to call this function

```
mov   rcx, fn
call  rcx
```

Under the System V ABI, if the return value is small enough to fit in a register, then the callee will put the return value into the accumulator. However, if the return value is of type MEMORY, then the caller is responsible for putting the address of a space in memory for the return value in rdi. Then, the callee puts the return value in the provided buffer, and may place the address of the buffer in eax for the caller.

In the case of MEMORY return type, the first argument moves to RSI.

A return value can use registers only if its ABI classification is not MEMORY. For common integer/pointer/float aggregates, that often means it must fit into at most two eightbytes, but size alone is not the whole rule

- INTEGER-class pieces use RAX, then RDX
- SSE-class pieces use XMM0, then XMM1
- If the type is classified as MEMORY, the caller passes a hidden pointer in RDI instead.

Consider a structure

```
struct A {
    long x,y,z;
};

A fn(int a) {}
```

Note that the class of the return value is MEMORY. `fn` is effectively called like this at the ABI level:

```
void fn(A* retbuf, int a);
```

A simple caller-side sketch looks like this:

```
sub     rsp, 24          ; make space for A
lea     rdi, [rsp]       ; hidden return buffer
mov     esi, 123         ; a = 123
call    fn
; now [rsp], [rsp+8], [rsp+16] hold x,y,z
```

Note that `lea` is *load effective address*.

```
; fn(A* retbuf, int a)
; RDI = retbuf
; ESI = a

movsxd  rax, esi
mov     [rdi],     rax       ; retbuf->x = a
lea     rcx, [rax+1]
mov     [rdi+8],   rcx       ; retbuf->y = a + 1
lea     rcx, [rax+2]
mov     [rdi+16],  rcx       ; retbuf->z = a + 2

mov     rax, rdi             ; return same pointer per ABI
ret
```

Note that `movsxd` is move with Sign-Extend Doubleword. It is a 64-bit mode (x86-64) instruction that copies a 32-bit (doubleword) source operand to a 64-bit (quadword) destination register while preserving its signed value by sign-extending the most significant bit.

Note: If a C++ object has a non-trivial copy constructor or non-trivial destructor, it is passed by invisible reference

If you define a constructor yourself, it is considered non-trivial, even if it doesn't do anything

- **IA-32e indirect addressing**: Consider an instruction with operands $r/m32$, $r32$. Up to this point, we have not dealt with operands that refer to a memory location. How do we construct the ModR/m byte when an operand is a memory location. Furthermore, a memory location is 64-bit... how do we encode this?

To deal with this issue, we must learn about how IA-32e does indirect addressing. We place the memory address into a register, and then use that register in the encoding. If the instruction is

```
mov   addr,eax
```

Then, we first move `addr` into a register, say esi:

```
mov   esi, addr
```

Then, the mov instruction we want is

```
mov   [esi], eax
```

The [...] syntax means memory indirection: treat the register(s) inside the brackets as an effective address (EA), and read/write memory at that address. So: `mov [esi], eax` means: store the 32-bit value in EAX into memory at address (zero-extended) ESI.

Recall the high order two bits of the ModR/m byte:

- **00:** Memory operand, no displacement (except special cases)
- **01:** Memory operand + 8-bit displacement
- **10:** Memory operand + 32-bit displacement (16-bit in 16-bit mode)
- **11:** Register operand (no memory access)

Thus, we no longer use mod = 11, in the case above, we use mod = 00. So, the reg bits are 000 (eax), and the r/m bits are 110 (esi), and the byte is therefore

$$0b00000110 = 0x06.$$

When a displacement is present, the value of that displacement (in bytes) comes after the ModR/m byte in the encoding (in hex).

- **Clearing memory**: Before we write to memory, we may need to clear what already exists there, for example if we have an assignment statement. Consider the memory

```
int data[500];
data[0] = -1;
data[18] = -92;
data[47] = 37;
```

Then, we must do

```
XOR    data[0], data[0]
```

However, there is no memory to memory MOV operation in IA-32e. So, we can instead clear out a register, and then move the contents of the cleared register to our memory location.

```
0  XOR    EAX,EAX
1  MOV    DATA[0], EAX
```

First observe that the XOR instruction has encoding $31C0$. Now, to handle the MOV instruction, we must place the address of `data[0]` in a register, then use indirect addressing. So, we do

```
0  MOV    RSI, &data
1  MOV    [ESI], EAX
```

The first MOV uses

```
0  REX.W + B8+ rd io MOV r64, imm64
```

The second MOV uses

```
0  89 /r MOV r/m32, r32
```

and the encoding is 8906.

- **Booleans**: We use an 8-bit register (low order 8) to store boolean values. In our language, a boolean can only be zero or one, so in that register, only the LSB is of concern. For example, we may use AL.
- **Code generation for relational operations**: Consider an expression

$$e_1 e_2 < .$$

Using our stack based model,

```
0  push   e1
1  push   e2
2  pop    eax
3  pop    ecx
4  cmp    ecx, eax
5  setl   al
```

- **Short circuiting**: The idea is that for two logical operands, we could jump over the evaluation of the second operand depending on the result of the first. For example, for e_1 and e_2, we have e_1 whose expression was evaluated and set AL with the above logic. Then,

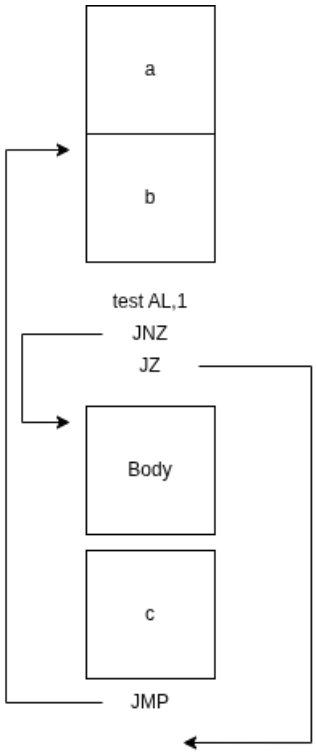
```
TEST AL,1
JZ ...
```

where the jump jumps over the evaluation of e_2.

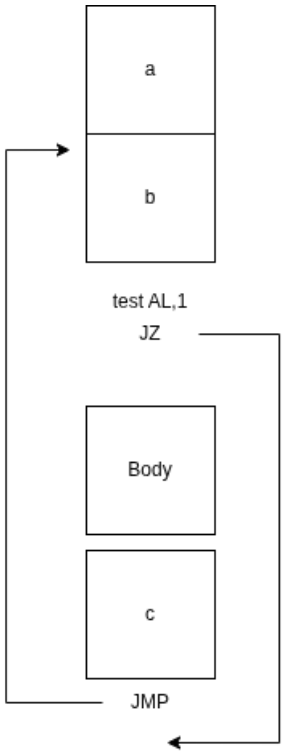
- **Looping**: Consider a for loop with the syntax

```
for (a; b; c) ..
```

Then, the code diagram is



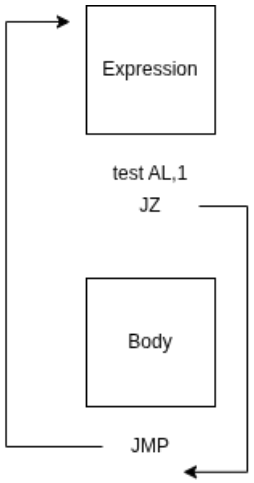
Alternatively,



If we have the while loop syntax

```
o  while (Expression) ...
```

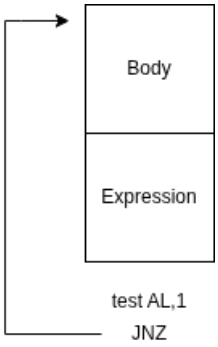
Then the code diagram would look like



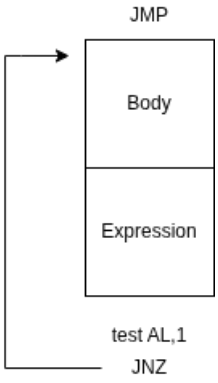
And for the do-while syntax

```
0  do ... while (expression);
```

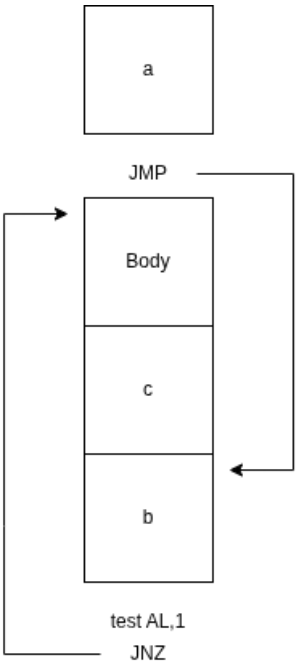
We have the code diagram



Notice the simplicity of the do-while code. Since this code is so simple, it is efficient to use this same structure for the while loop as well. To achieve this, we simply place an unconditional jump before the body's code, straight to the expression. This jump will happen only once, before the loop begins. Thus, a `while` loop has the code diagram



We can do the same for a for loop.



Thus, all loops are usually `do-while`s under the hood.

4.1 Functions

- **Register arguments**: Under the x86-64 System V ABI, the first six integer or pointer arguments are passed in registers:
  - rdi
  - rsi
  - rdx
  - rcx
  - r8
  - r9

  Floating-point arguments go in xmm0 through xmm7 in the usual cases. If there are more arguments than available argument registers, the extras are passed on the stack by the caller. So, most small function calls never need stack-passed arguments at all.

- **Stack frames (System V ABI)**: In the System V ABI, a stack frame is the portion of the stack a function uses for its own execution state while it runs. In the calling-functions context, the frame exists to support argument passing beyond register capacity, storage for local variables, saved registers, temporary spill space, and a stable place to return to when the function finishes.

  For x86-64 System V specifically, the calling convention defines who places arguments where, what must be preserved across calls, and what stack alignment rules must hold at a call site. That is what gives stack frames their shape.

  Suppose that A calls function B, at a high level:

  1. A prepares arguments.
  2. A ensures the stack is aligned correctly.
  3. A executes call B.
  4. The call instruction pushes the return address onto the stack and jumps to B.
  5. B runs, possibly building its own stack frame.
  6. B returns with ret, which pops the return address and jumps back to A.

  So the stack frame of B begins only after control has entered B.

  A stack frame is the region of the stack belonging to one function call. That region may contain:
  - the return address
  - the saved old frame pointer
  - local variables
  - spilled registers
  - saved callee-saved registers
  - temporary storage

- **rsp, rbp**: rsp and rbp are both stack-related registers, but they play different roles.

- **rsp**: is the stack pointer. It always tracks the current top of the stack. Since the x86-64 stack grows downward, pushing something onto the stack decreases rsp, and popping something increases it. So rsp is the register that reflects the stack's live, current position at every moment.
- **rbp**: rbp is traditionally the frame pointer. It does not have to be used, but when it is used, it serves as a stable reference point for the current function's stack frame.

The issue is that rsp can move during the function. If the function pushes registers, allocates locals, or reserves temporary space, the value of rsp changes. That makes it less convenient as a fixed reference point. That is why many functions do this at the beginning:

```
0  push rbp
1  mov rbp, rsp
2  sub rsb,32
```

- push rbp saves the caller's frame pointer
- mov rbp, rsp makes rbp point to the base of the new frame
- sub rsp, 32 reserves 32 bytes for local storage

So the usual mental model is:

- rbp = stable anchor for this frame
- rsp = moving top of stack

The stack might look like this

```
0  higher addresses
1  -------------------------
2  arguments passed on stack
3  return address
4  saved old rbp      <- rbp
5  local variable 1
6  local variable 2
7  temporary space
8  -------------------------
9  lower addresses    <- rsp
```

- **Restoration**: At the end of the function, the epilogue often looks like this:

```
0  mov rsp, rbp
1  pop rbp
2  ret
```

Or equivalently,

```
0  leave
1  ret
```

– mov rsp, rbp discards the local variable space all at once

Other registers are handled according to the ABI's save convention. The central question is: Who is responsible for preserving a register's value across a function call?

1. **Caller-saved registers**: These are also called volatile or call-clobbered registers.

   If the caller has a value in one of these registers and still needs that value after making a function call, the caller must save it before the call and restore it afterward.

   In System V x86-64, the general-purpose caller-saved registers are:

   – rax, rcx, rdx, rsi, rdi
   – r8, r9, r10, r11

   So if a function f is using, say, r10, and then it calls g, it must assume g is allowed to overwrite r10.

   ```
   mov r10, 123
   push r10
   call g
   pop r10
   ```

2. **Callee-saved registers**: These are also called nonvolatile or call-preserved registers. If the callee wants to use one of these, then the callee must save the old value and restore it before returning.

   In System V x86-64, the general-purpose callee-saved registers are:

   – rbx, rbp
   – r12, r13, r14, r15

- **Stack alignment**: In x86-64, stack alignment means the stack pointer must be positioned on a specific byte boundary at certain times, usually a 16-byte boundary. So,

$$\text{rsp} \equiv 0 \pmod{16}.$$

Thus, the stack pointer is a multiple of 16. In normal x86-64 function calling, the important rule is this:

– Before a call instruction, the caller must arrange things so the callee sees the expected alignment.
– Since call pushes an 8-byte return address, the stack shifts by 8 bytes when control enters the called function.

Under the System V AMD64 ABI used on Linux and most Unix-like systems, the usual convention is:

– Just before call, the caller makes rsp 16-byte aligned
– After call pushes the return address, inside the callee at entry:

$$\text{rsp} \equiv 8 \pmod{16}.$$

The general idea is:

- Check what rsp mod 16 currently is.
- Subtract enough bytes so that before call, rsp % 16 == 0.
- Restore it afterward with add rsp, same_amount.

So,

$$N = \text{rsp mod } 16 = \text{rsp } \&0xF,$$

since $N < 16$ and 16 is a power of 2.

- **Caller saved registers**: In x86-64 assembly, caller-saved (also known as volatile or call-clobbered) registers are those that a called function (the callee) is permitted to modify or overwrite without restoring their original values. If the calling function (the caller) needs the data in these registers to remain intact after a function call, it is responsible for saving them (typically by pushing them onto the stack) before making the call

  Those registers are

  - RAX, RCX, RDX, RSI, RDI, R8, R9, R10, R11
  - XMM0 – XMM15

- **Callee saved registers**: In x86-64 assembly, callee-saved registers (also known as non-volatile or call-preserved registers) are those whose values must be preserved across a function call. If a called function (the callee) needs to use one of these registers, it is responsible for saving the original value (typically by pushing it onto the stack) and restoring it before returning to the caller.

  Those registers are the ones not marked as caller saved,

  - RBX, RSP, RBP, R12, R13, R14, R15

- **Caller prologue / epilogue (stack based)**: Suppose we wanted to use a stack model for the caller / callee. The prologue / epilogue for the caller would look something like

  Prologue:

  1. Push arguments onto the stack in reverse order
  2. Save caller saved registers

  Epilogue:

  1. Adjust stack pointer for arguments
  2. Restore call-clobbered registers

  Note: In practice, some arguments (small arguments like pointers and integers) can go into registers, while others (like large arguments) go into the stack memory.

- **Callee prologue / epilogue**: Then, for the callee:

Prologue:

1. Push rbp onto the stack
2. `mov rbp, rsp`
3. Make room for local vars (`sub rsp, k`)
4. Initialize local vars

Epilogue:

1. `mov rsp, rbp`
2. `pop rbp`
3. `ret`

- **Spilling**: Spilling means the compiler or code generator has to temporarily move a value from a register to memory, usually the stack, because there are not enough registers available. That memory location is called a **spill slot**.
- **Stack frame**: The stack frame for a function looks something like

Figure 3.3: Stack Frame with Base Pointer

Position	Contents	Frame
8n+16(%rbp)	memory argument eightbyte n	Previous
	...	
16(%rbp)	memory argument eightbyte 0	
8(%rbp)	return address	Current
0(%rbp)	previous %rbp value	
-8(%rbp)	unspecified	
	...	
0(%rsp)	variable size	
-128(%rsp)	red zone	

- **Using the stack for return values**: For large structs or objects, the caller usually reserves space and passes the address of that space to the callee. The callee writes the result there. So,

```
Big make_big();
```

May get implemented more like

```
void make_big(Big* hidden_return_slot);
```

The caller does something like:

- reserve stack space for the returned object
- pass the address of that space to the callee
- callee fills it in
- caller reads the result from that stack space

So the returned value lives in stack memory owned by the caller.

```
sub rsp, 32          ; reserve space for returned object
lea rdi, [rsp]       ; pass pointer to return slot
call make_big
; now [rsp]..[rsp+31] contains the returned struct
```

- **Note: Size qualifiers**: Assembly size qualifiers tell the assembler how large the memory operand is.
  - byte ptr = 1 byte = 8 bits
  - word ptr = 2 bytes = 16 bits
  - dword ptr = 4 bytes = 32 bits
  - qword ptr = 8 bytes = 64 bits
  - tword ptr = 10 bytes, often for x87 extended precision values
  - xmmword ptr = 16 bytes
  - ymmword ptr = 32 bytes
  - zmmword ptr = 64 bytes

  It is not a cast. It is more like a type annotation for the memory operand.

```
mov dword ptr [rdi], 5
```

  says

  - take the address in rdi
  - look at the memory there
  - treat that destination as a 4-byte object
  - store 5 into it

Optimizations

- **Unary negation**: One very simple optimization that we can do when parsing is to collapse the unary negation operation on an integer constant. For example,

  becomes simply -5.

- **Constant folding**: Constant folding is a compiler optimization where the compiler evaluates expressions made entirely of constants at compile time instead of generating code to compute them at run time. For example, with

```
x := 3 + 4 * 2;
```

  the compiler can compute 4 * 2 = 8, then 3 + 8 = 11, and replace the expression with

```
x := 11
```

  This is called "folding" because the compiler collapses a constant expression subtree into a single constant node.

- **Dead code elimination**: Dead code elimination is an optimization that removes code which cannot affect the program's observable behavior.
  - **Unreachable code**: Suppose we have

```
if (false) {
    x := 5;
}
```

    The body can never be executed, so it is dead code.
  - **Executed but useless**: Consider

```
x := 3 + 4;
x := 9;
```

    The first assignment is never read before being overwritten, so the first assignment is dead.

  So dead code elimination is broader than just "removing unreachable statements." It also includes removing useless computations and stores.

- **Common subexpression elimination (CSE)**